

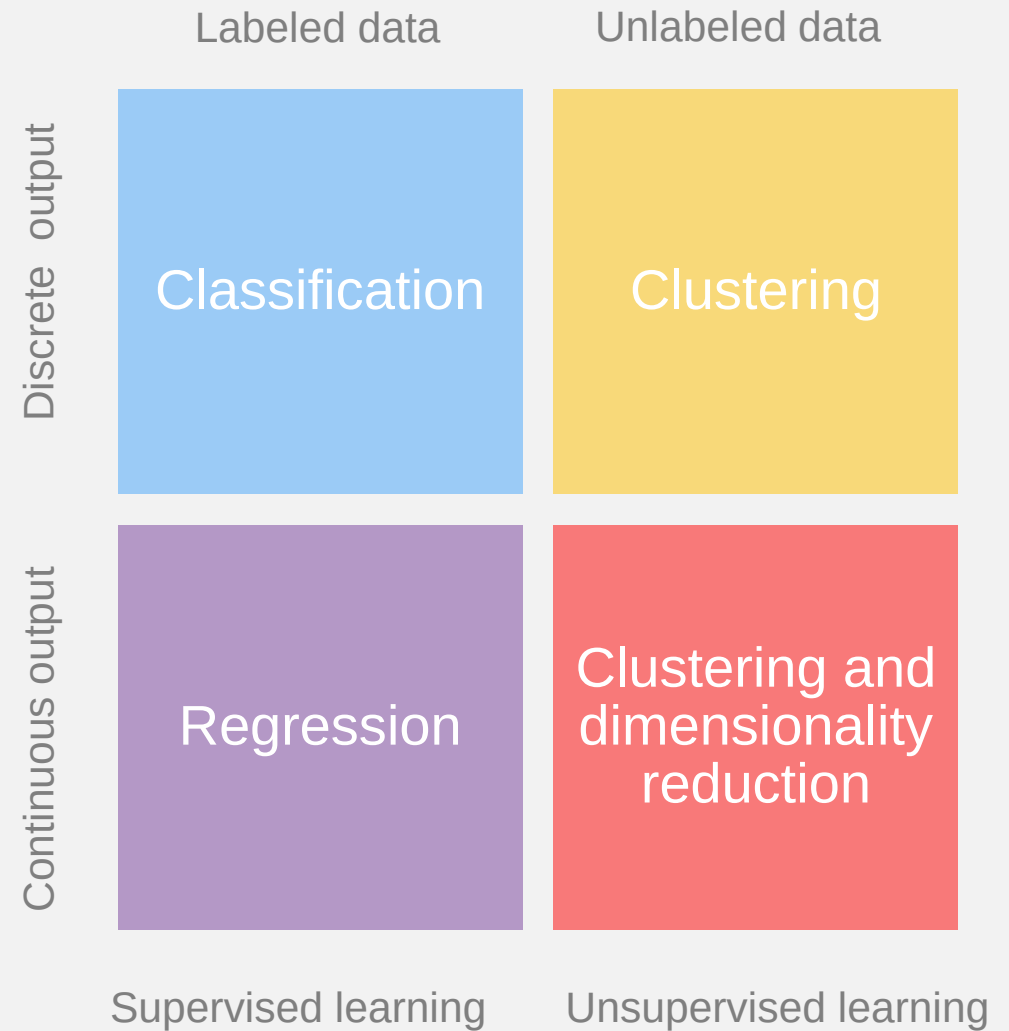
Supervised Machine Learning (*Classifiers*)

Computer Vision and Artificial Intelligence

Dr Varun Ojha

varun.ojha@ncl.ac.uk

School of Computing
Newcastle University



This is draft version do not print or if you must, consider printing 6 slides per page both sides

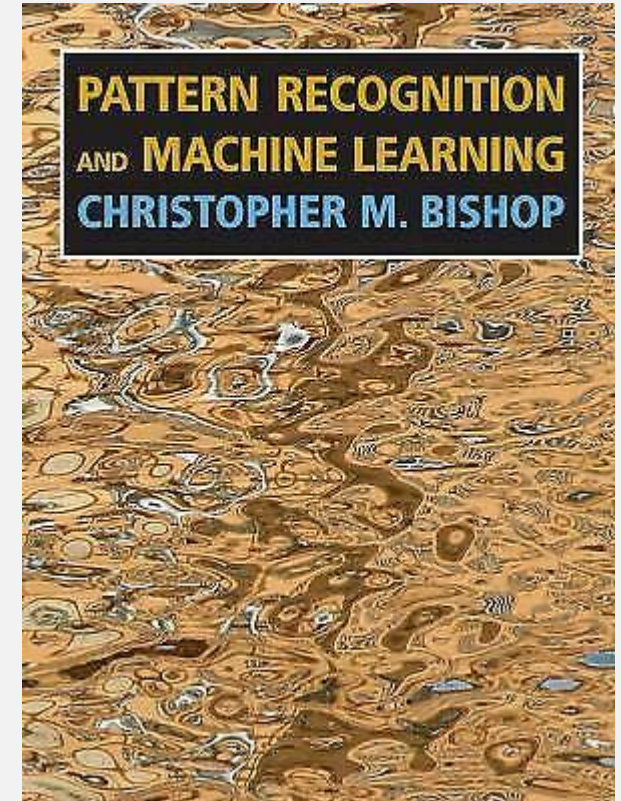
Learning objectives

By the end of this week, you will be able to

- Learn concepts of classification in supervised learning
- Understand different type algorithms for data classification
- Distinguish features of different classifier evaluation methods
- Apply and evaluate classifiers problem – *in practical session*

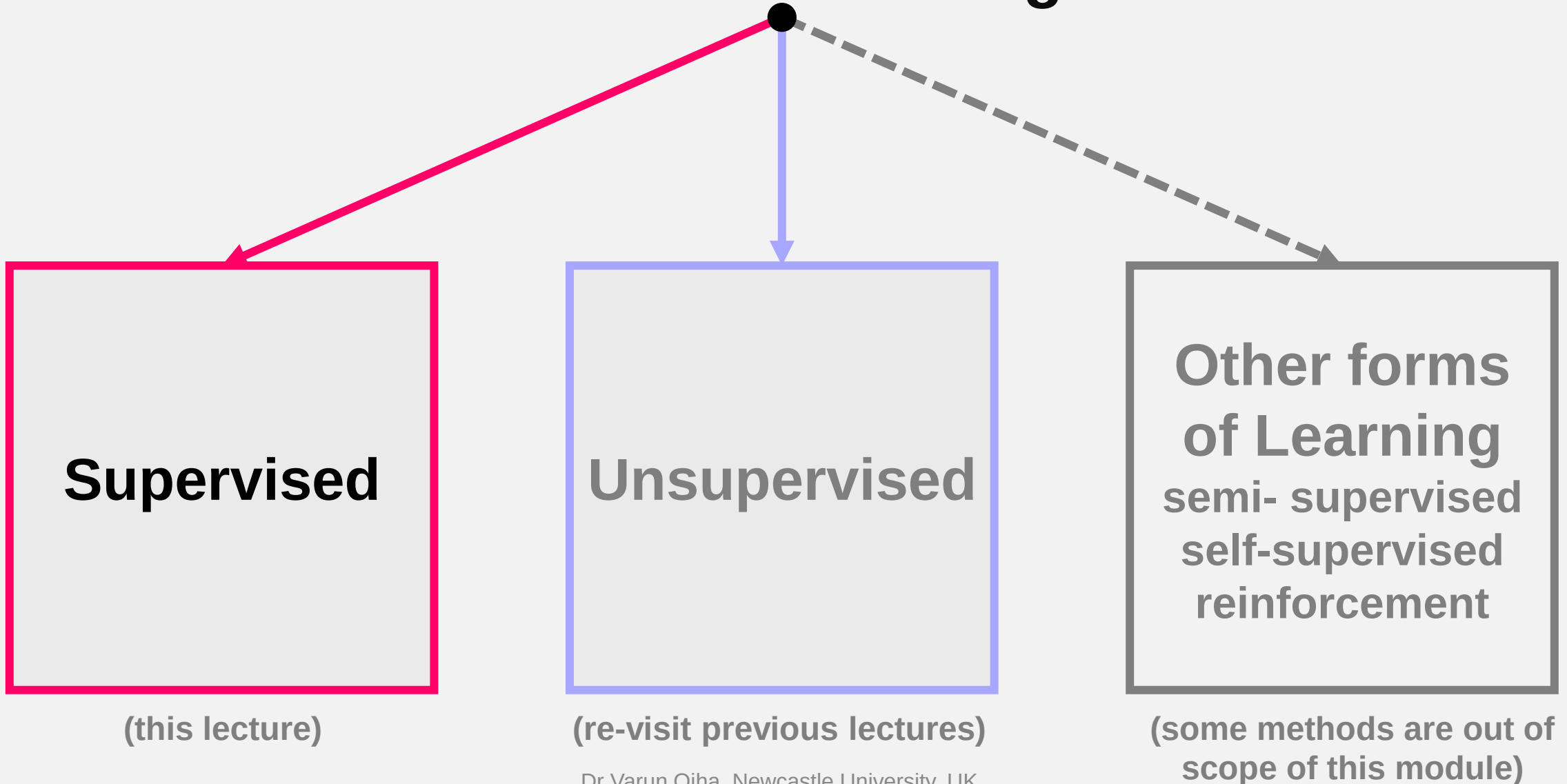
Content

- **Part 1: Introduction**
- **Part 2: Classifiers**
 - K-Nearest Neighbour
 - Naïve Bayse Classifier
 - Decision Tree and Random Forest
 - Support Vector Machines
- **Part 3: Model Evaluation**
 - Accuracy
 - Confusion Metrix
 - ROC/AUC/Model Comparison
- **Part 4: Practical Exercise**

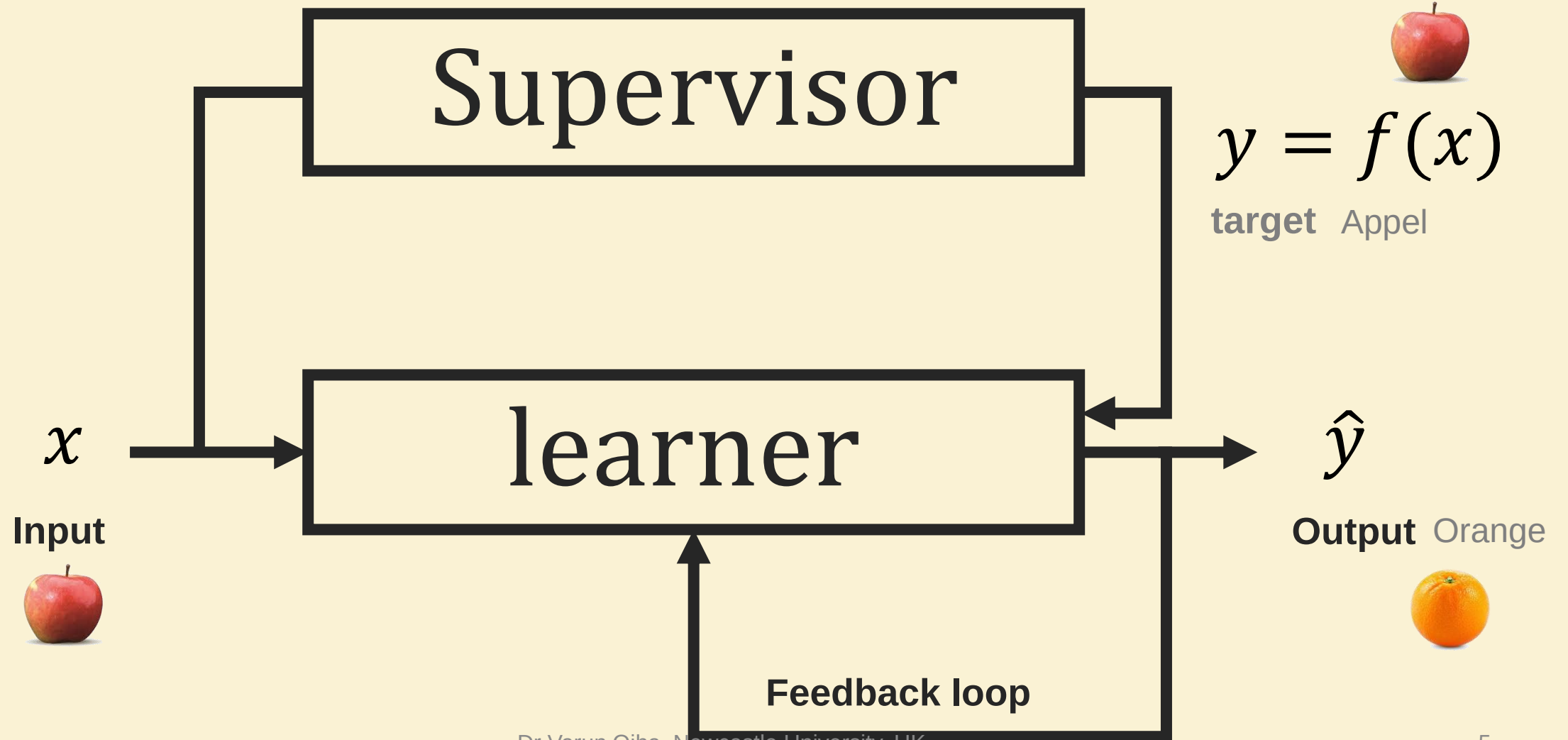


<https://www.microsoft.com/en-us/research/publication/pattern-recognition-machine-learning/>

Machine Learning



Supervised





Regression and Classification

Class/Target attribute

	#	Inputs Attributes (Independent)		Target/Class/Output Attributes (Dependent)
		A1	A2	A3
Records	Ex. 0	$A1_0$	$A2_0$	$A3_0$
	Ex. 1	$A1_1$	$A2_1$	$A3_1$
	Ex. 2	$A1_2$	$A2_2$	$A3_2$
	Ex. 3	$A1_3$	$A2_3$	$A3_3$
	Ex. 4	$A1_4$	$A2_4$	$A3_4$
	Ex. 5	$A1_5$	$A2_5$	$A3_5$
	Ex. 6	$A1_6$	$A2_6$	$A3_6$
	Ex. 7	$A1_7$	$A2_7$	$A3_7$
	Ex. 8	$A1_8$	$A2_8$	$A3_8$
	Ex. 9	$A1_9$	$A2_9$	$A3_9$

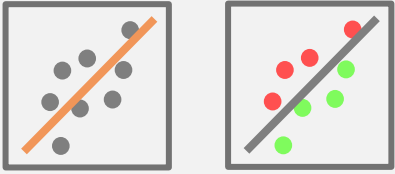
**Target (Class)
Attributes (A3)**

Regression

**Continuous
(Numerical)**
labeled data

Classification

**Discrete
(Categorical)**
labeled data



Tasks: Regression and Classification

Continuous labeled data

#	Inputs (X)		Target (Y)	
	Area (m ²)	Distance(mile)	Price (£Bn)	
Ex. 0	76.85	17.27	0.15	
Ex. 1	76.97	19.54	0.5	
Ex. 2	77.10	18.51	0.76	
Ex. 3	85.28	46.09	0.23	
Ex. 4	85.42	35.83	0.6	
Ex. 5	88.02	2.59	0.67	
Ex. 6	77.25	6.34	0.89	
Ex. 7	77.49	6.98	0.2	
Ex. 8	85.81	12.18	0.55	
Ex. 9	98.81	2.18	9.45	

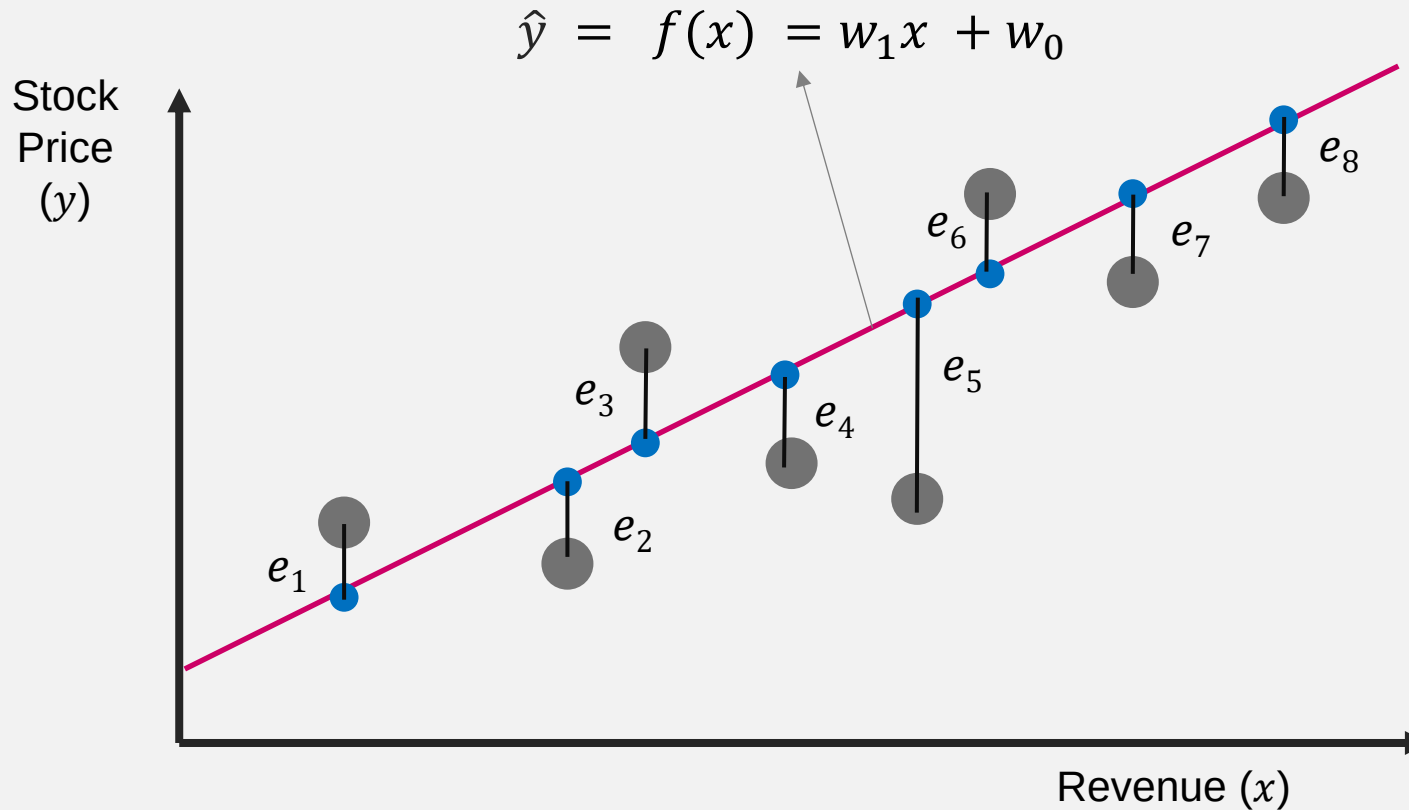
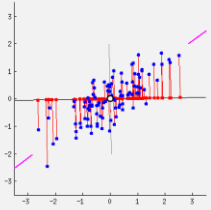


Discrete labeled data

#	Inputs (X)		Class (Y)	
	Length (cm)	Weight (kg)	Sales	
Ex. 0	23.2	3.2	Good	
Ex. 1	70.9	19.5	Bad	
Ex. 2	60.5	18.51	Bad	
Ex. 3	24.5	4.6	Good	
Ex. 4	110.0	35.83	Bad	
Ex. 5	23.8	3.7	Good	
Ex. 6	25.8	4.5	Good	
Ex. 7	24.7	4.9	Good	
Ex. 8	85.8	25.6	Bad	
Ex. 9	78.8	20.33	Bad	



Regression



- ✓ Best Fit
- ✓ Find the line (parameters of a line equation) that minimize the norm of the y errors
- ✓ (sum of the squares)

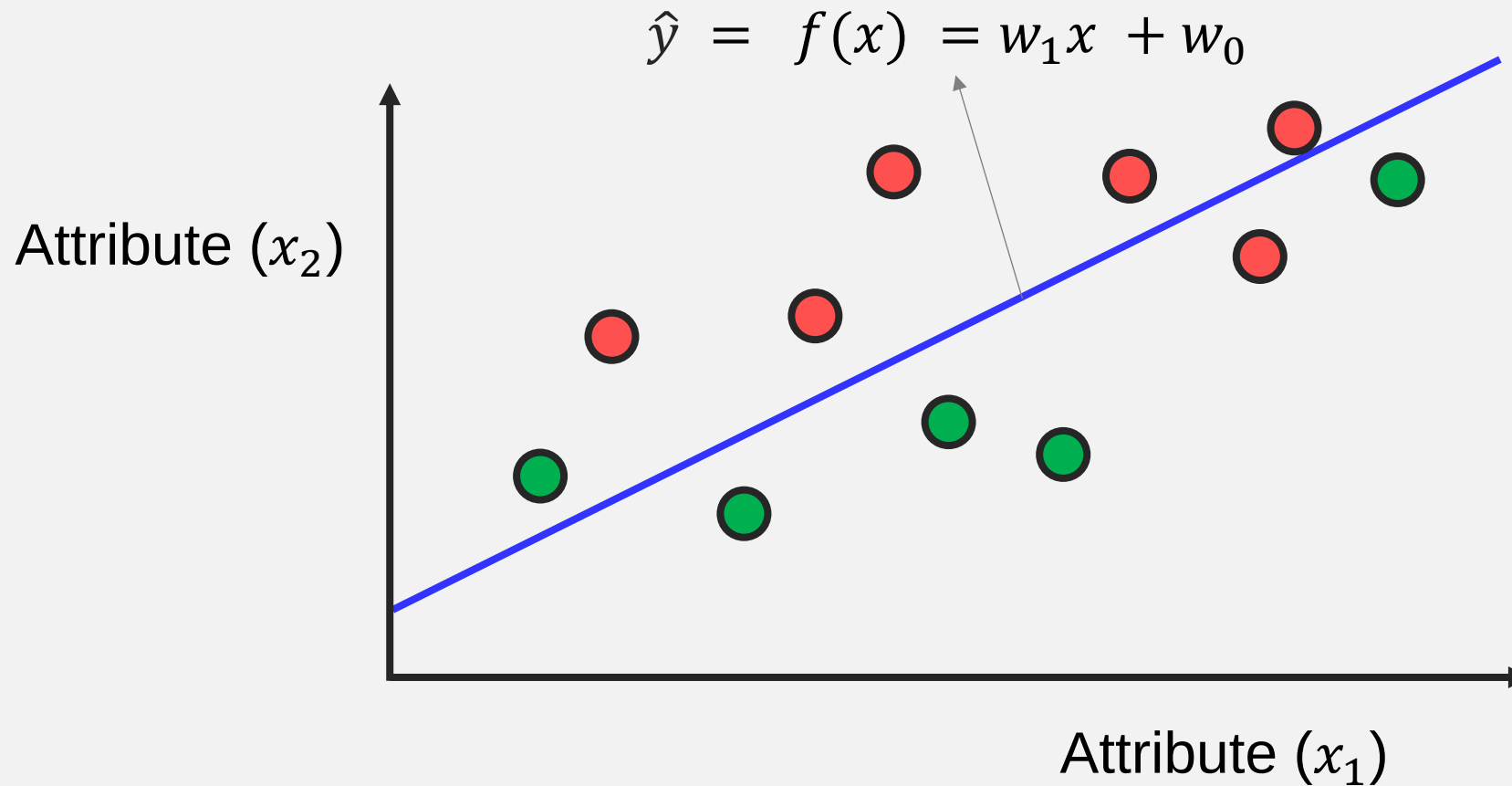
Error

$e_i = \hat{y}_i - y_i$

$$e = \sum_{i=1}^8 (\hat{y}_i - y_i)^2$$



Classification



- ✓ Best Fit
- ✓ Find the line (parameters of a line equation) that minimize the error (misclassification) rate

$$e = \frac{1}{n} \sum_{i=1}^n \hat{y}_i \neq y_i$$

Part 2/a

Type of Classifiers

- Instance Based Classifier
- Tree Based Classifier
- Naive Bays Classifier
- Support Vector Machine



Instance based classifier

Simplest form of learning: ***rote/memory learning***

- ✓ Training instances are searched for instance that most closely resembles new instance
- ✓ The instances themselves represent the knowledge
- ✓ Also called instance-based learning
- ✓ Memorizes entire training data and performs classification only if attributes of record match one of the training examples exactly

Similarity function (distance) defines what's "learned"

Instance-based learning is *lazy* learning

Nearest Neighbor:

Uses "closest" points (nearest neighbors) for performing classification

nearest-neighbor (1-NN)

k-nearest-neighbor (k-NN)

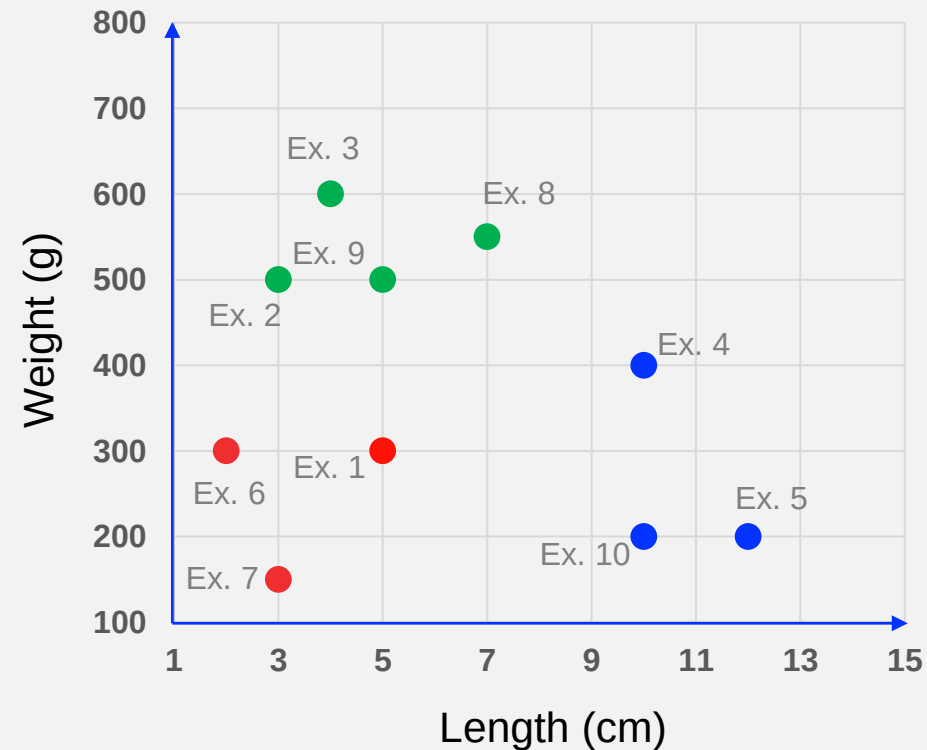


K-Nearest Neighbor

Instance-based algorithm (Lazy Lerner)

Discrete labeled data

#	Inputs		Target
	Length (cm)	Weight (g)	Fruit
Ex. 1	5	300	Orange
Ex. 2	3	500	Apple
Ex. 3	4	600	Apple
Ex. 4	10	400	Mango
Ex. 5	12	200	Mango
Ex. 6	2	300	Orange
Ex. 7	3	150	Orange
Ex. 8	7	550	Apple
Ex. 9	5	500	Apple
Ex. 10	10	200	Mango



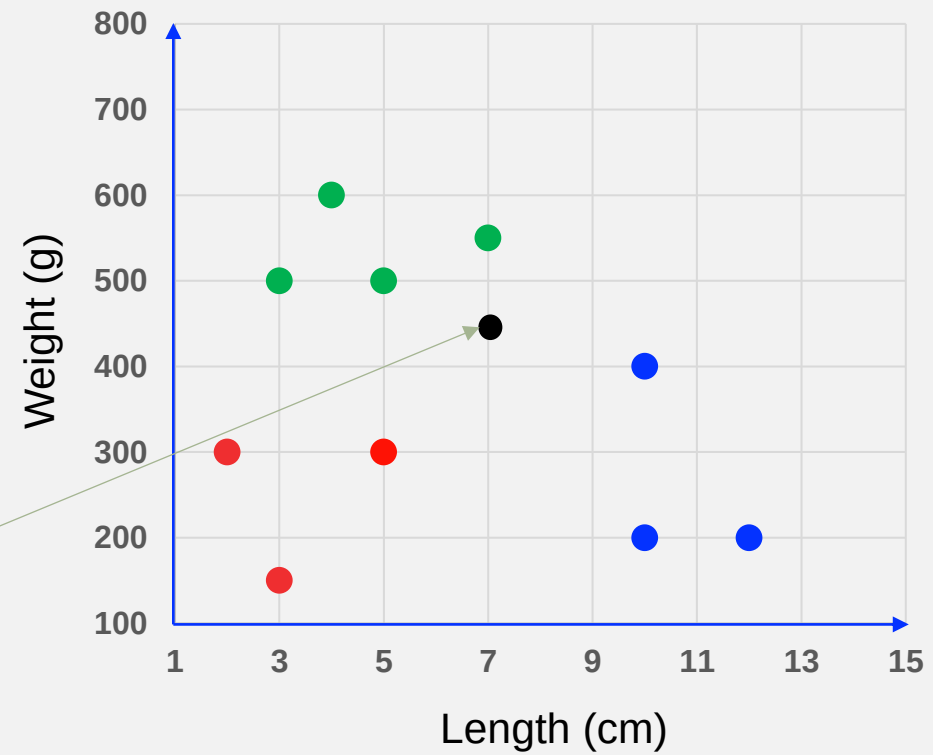


K-Nearest Neighbor

Question: What is the label of a new instance?

Unseen instance:

#	Inputs		Target
	Length (cm)	Weight (g)	Fruit
Ex. 11	7	450	?



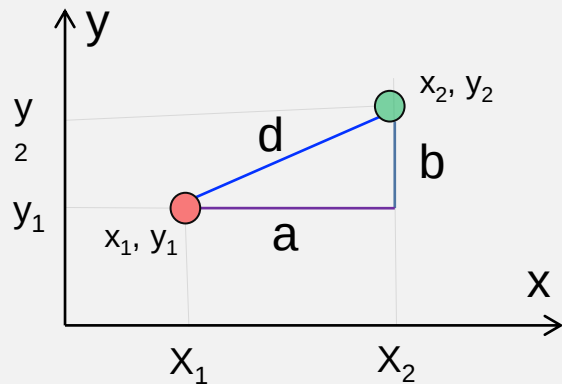


K-Nearest Neighbor

Answer: Find the K-Nearest Neighbors

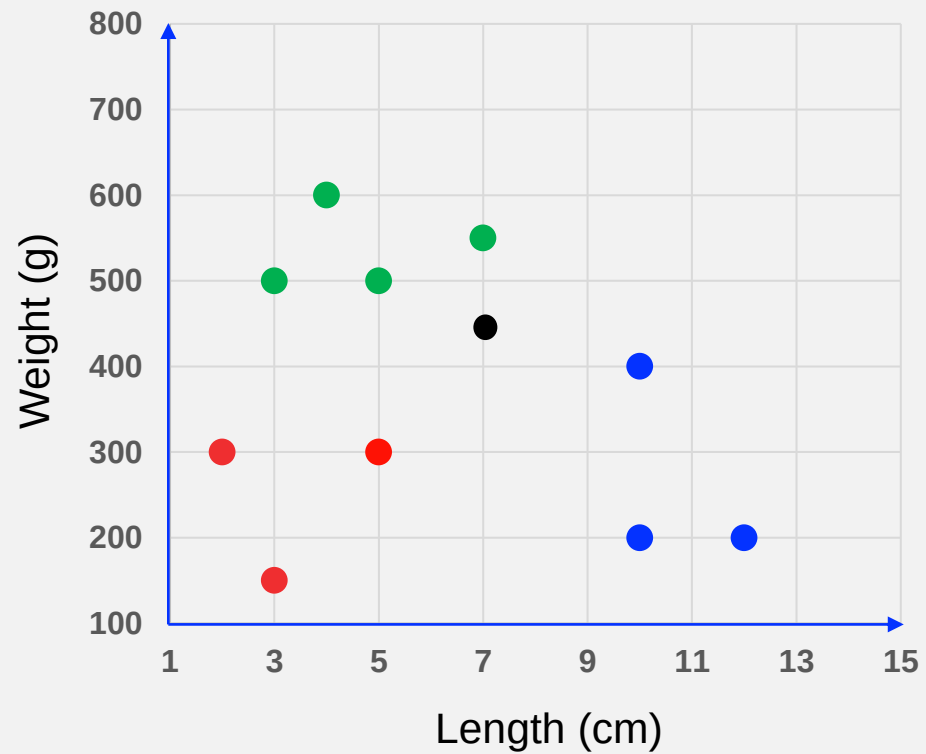
How we do that?

Euclidean distance:



$$d^2 = a^2 + b^2$$

$$d(p_1, p_2) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$



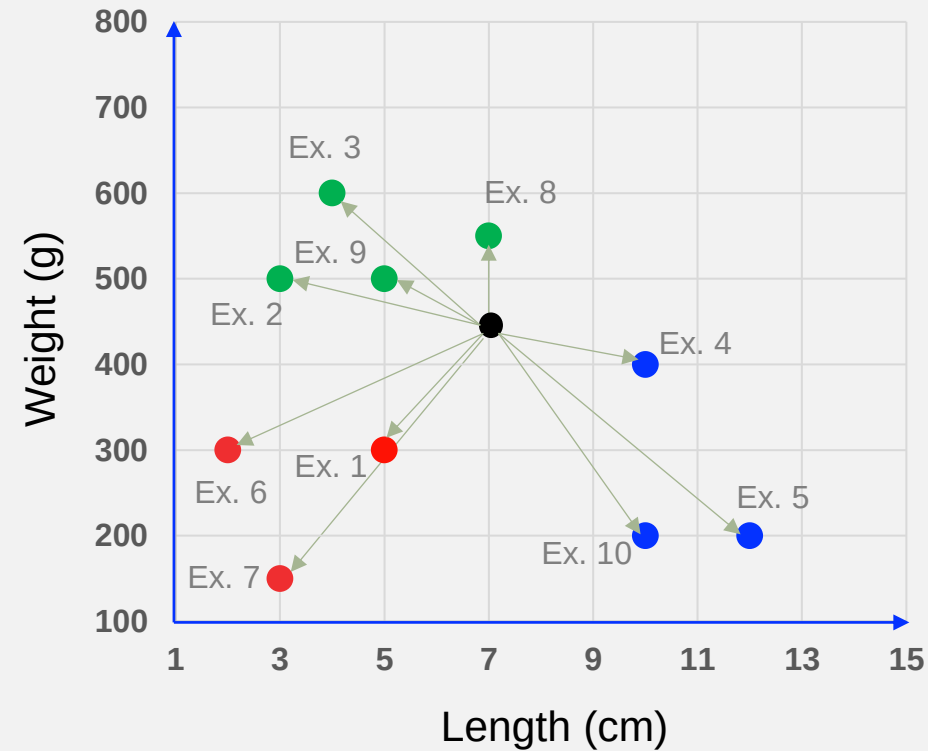


K-Nearest Neighbor

Which neighbor is the nearest?

Euclidean distance of all examples:

EX. 1	● → ●	150.02
EX. 2	● → ●	50.16
EX. 3	● → ●	150.03
EX. 4	● → ●	50.09
EX. 5	● → ●	250.05
EX. 6	● → ●	150.08
EX. 7	● → ●	300.03
EX. 8	● → ●	100.00
EX. 9	● → ●	50.04
EX. 10	● → ●	250.02

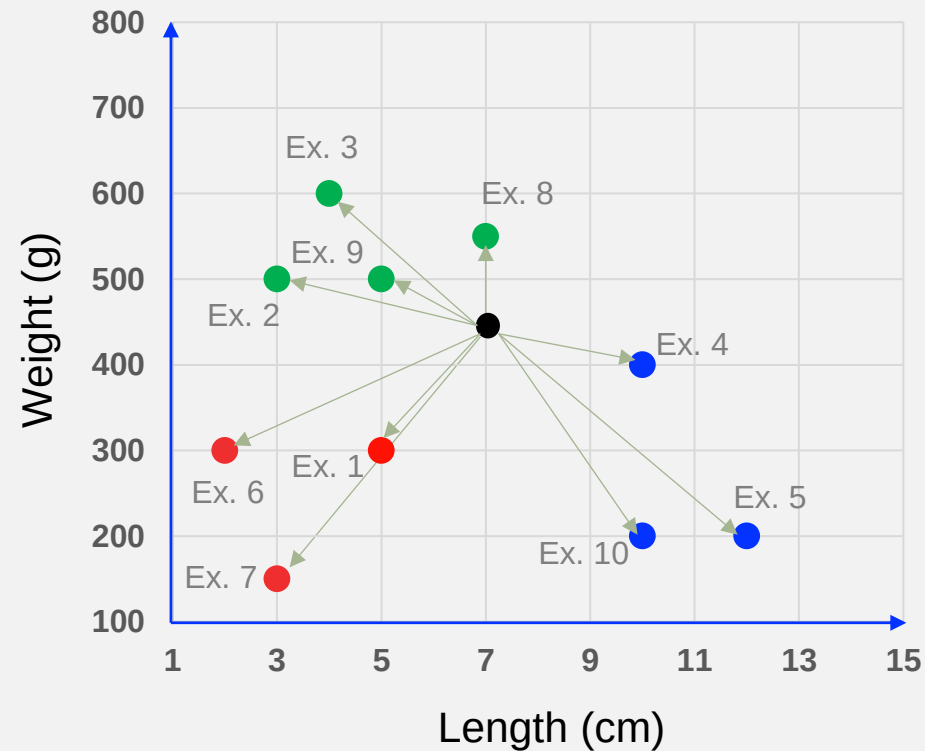




K-Nearest Neighbor

Shortest distance first?

EX. 9	● → ●	50.04
EX. 4	● → ●	50.09
EX. 2	● → ●	50.16
EX. 8	● → ●	100.00
EX. 3	● → ●	150.03
EX. 1	● → ●	150.02
EX. 6	● → ●	150.08
EX. 10	● → ●	250.02
EX. 5	● → ●	250.05
EX. 7	● → ●	300.03





K-Nearest Neighbor

Let's arbitrarily set $K = 3$

Ex. 9 ● → ● 50.04 Neighbor 1

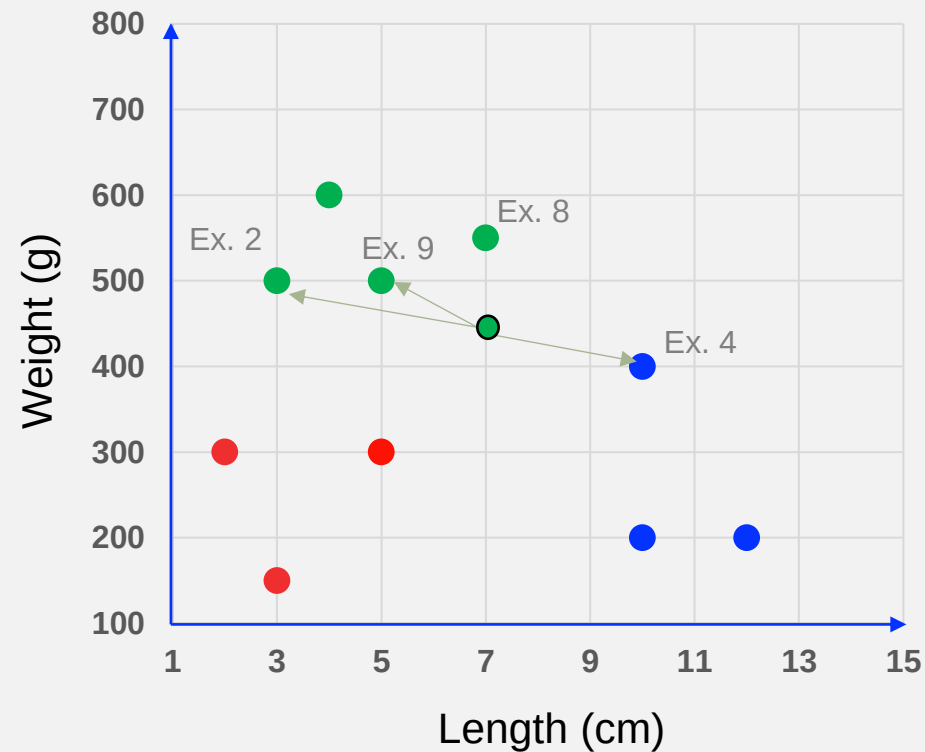
Ex. 4 ● → ● 50.09 Neighbor 2

Ex. 2 ● → ● 50.16 Neighbor 3

Majority of neighbors have label
"Green- Apple"

Unseen instance:

#	Inputs		Target
	Height (cm)	Weight (g)	Fruit
Ex. 11	7	450	A





Improving K-Nearest Neighbor

Optimizing:

K value

Distance metric

Taking weighted majority vote

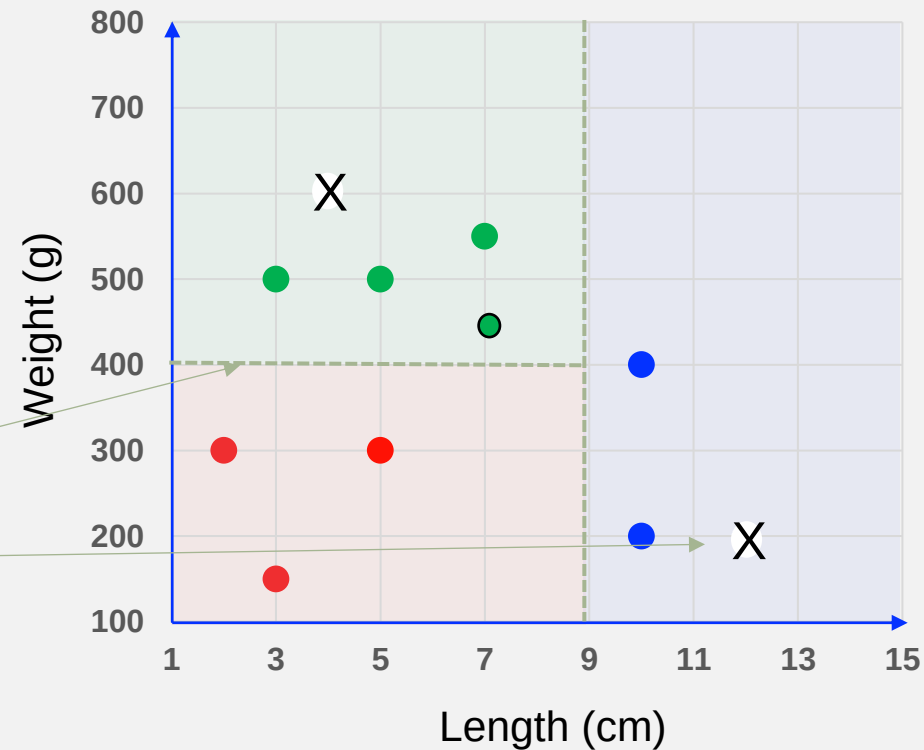
Feature scaling

Improving classification speed:

Partitioning data space (K-D Tree)

Reduce training data

Reducing feature dimension





Improving K-Nearest Neighbor

Normalization of Attributes (Scaling)

i

j

#	X Inputs Attributes (Independent)	
	A1	A2
Ex. 0	$A1_0$	$A2_0$
Ex. 1	$A1_1$	$A2_1$
Ex. 2	$A1_2$	$A2_2$
Ex. 3	$A1_3$	$A2_3$
Ex. 4	$A1_4$	$A2_4$
Ex. 5	$A1_5$	$A2_5$
MIN	$A1_{\text{MIN}}$	$A2_{\text{MIN}}$
MAX	$A1_{\text{MAX}}$	$A2_{\text{MAX}}$

Scaling attributes between $[a, b]$

$$A'_i = a + \frac{(A_{ij} - A_{i(\text{MIN})}) (a - b)}{A_{i(\text{MIN})} - A_{i(\text{MAX})}}$$

Part 2/b

Bayesian Classifier

Probability



Probability of choosing either 0 or 1

$$P \text{ (either 0 or 1)} = \frac{1 \text{ (favoured outcomes)}}{2 \text{ (all possible outcomes)}}$$

Probability of choosing even number

$$P(\text{EVEN number between 1 and 10}) = \frac{5}{10}$$

Bayes Theorem

$$P(H | E) = \frac{P(E | H) P(H)}{P(E)}$$



Thomas Bayes
(1701 – 1761)

Bayes Theorem

The diagram shows the Bayes Theorem equation with four labels and arrows pointing to specific parts of the formula:

- likelihood**: points to $P(E | H)$
- prior**: points to $P(H)$
- posterior**: points to $P(H | E)$
- normalising constant**: points to $P(E)$

$$P(H | E) = \frac{P(E | H) P(H)}{P(E)}$$

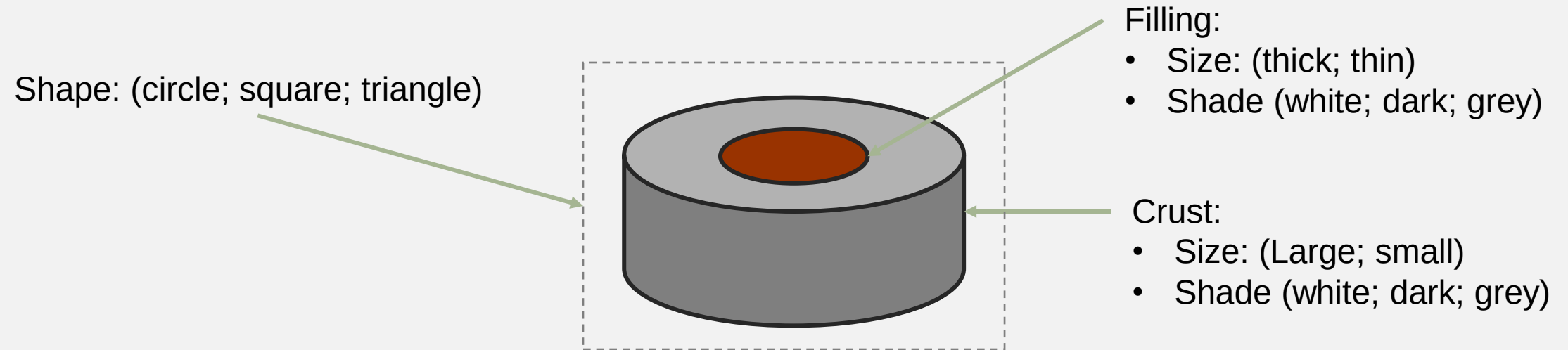
Where H and E are events

$P(H | E)$ is a conditional probability, the likelihood of H given E is true.

$P(E | H)$ is a conditional probability, the likelihood of E given H is true.

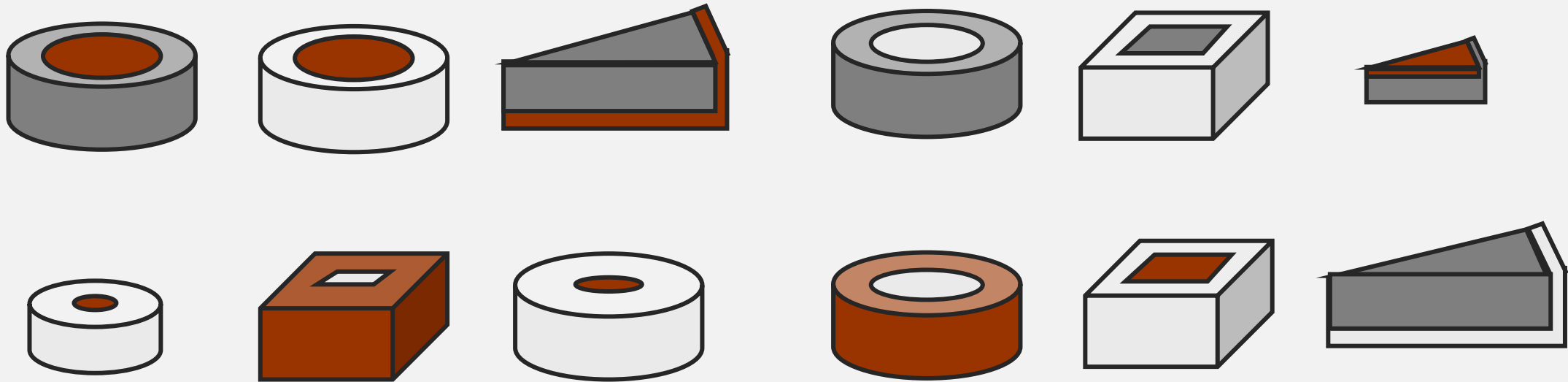
$P(H)$ and $P(E)$ are probabilities of observing H and E

Training data



Example Source: Kubat, M., 2017. *An introduction to machine learning* (Vol. 2). Cham, Switzerland: Springer International Publishing.

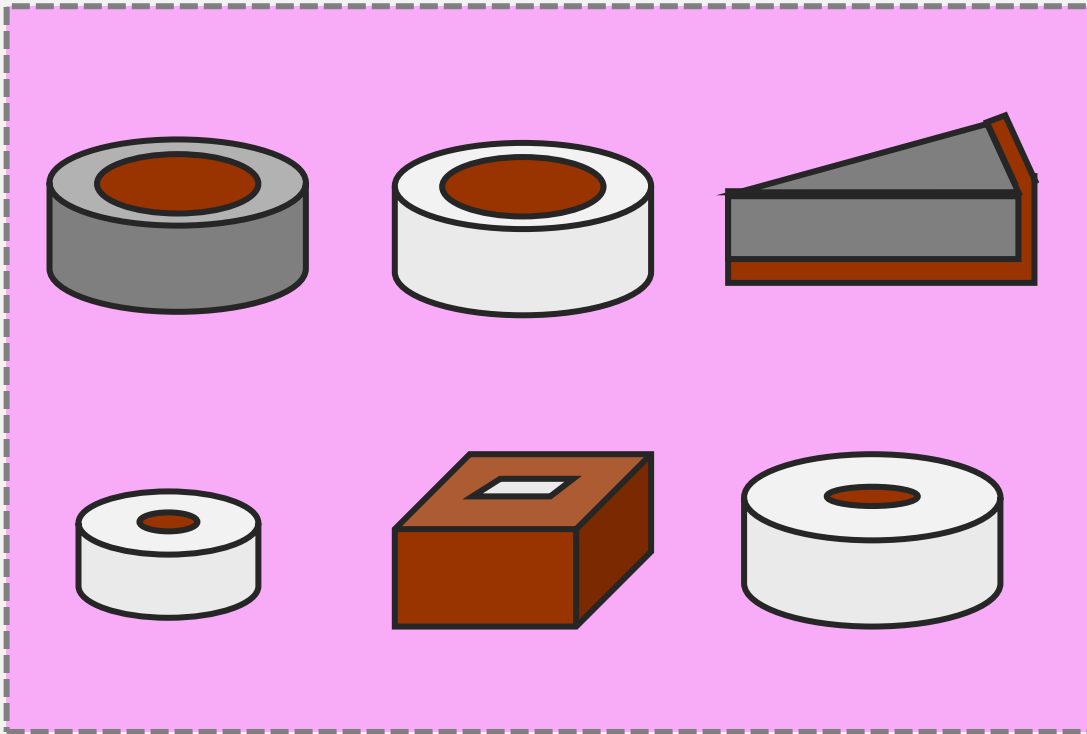
Cakes (Some data)



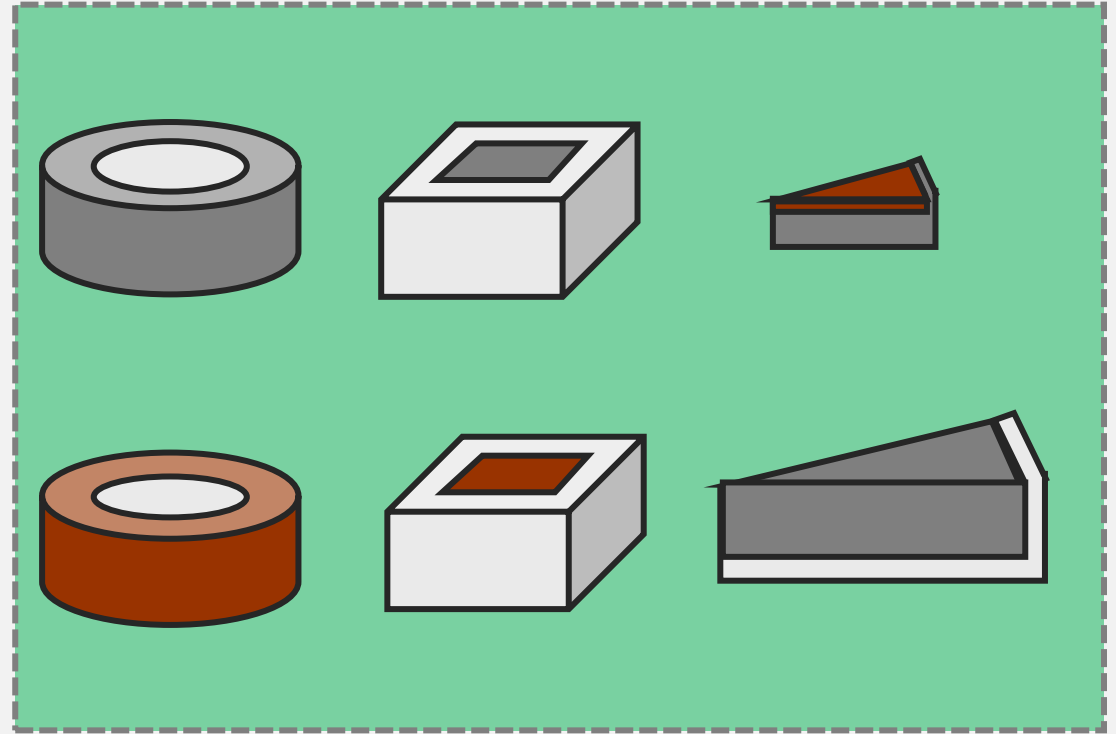
Example Source: Kubat, M., 2017. *An introduction to machine learning* (Vol. 2). Cham, Switzerland: Springer International Publishing.

Training data

I **LIKE** these types of cake (positives)



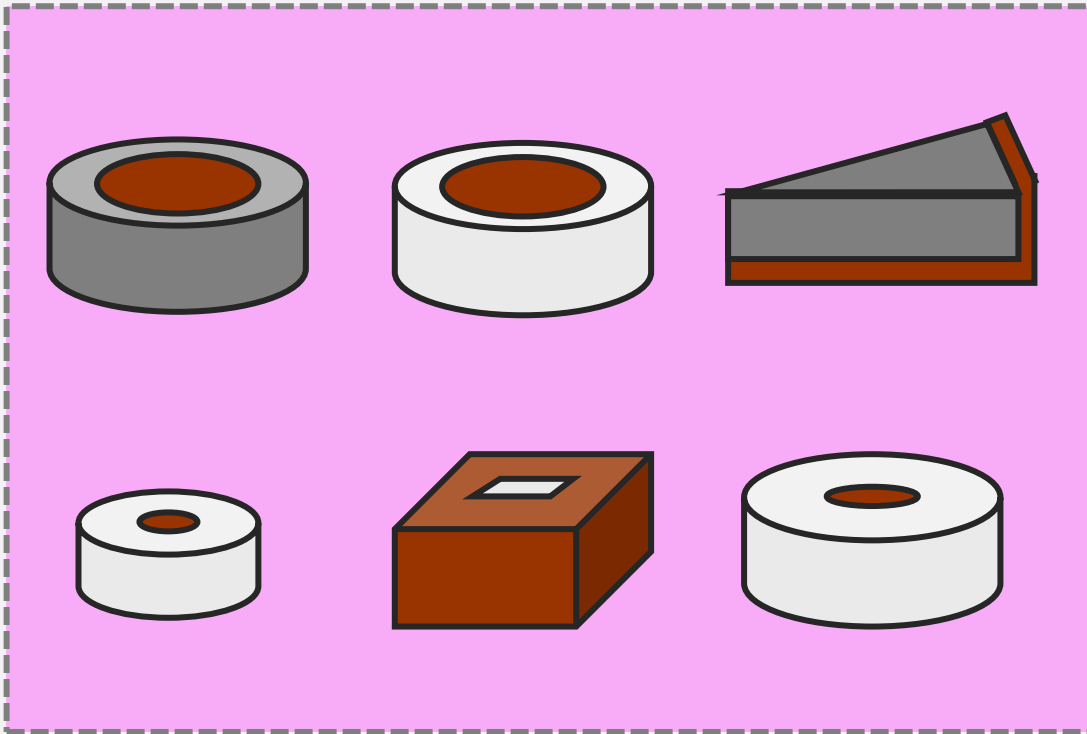
I **DO NOT LIKE** these types of cake (negatives)



Example Source: Kubat, M., 2017. *An introduction to machine learning* (Vol. 2). Cham, Switzerland: Springer International Publishing.

Training data: Positive example

I **LIKE** these types of cake (positives)

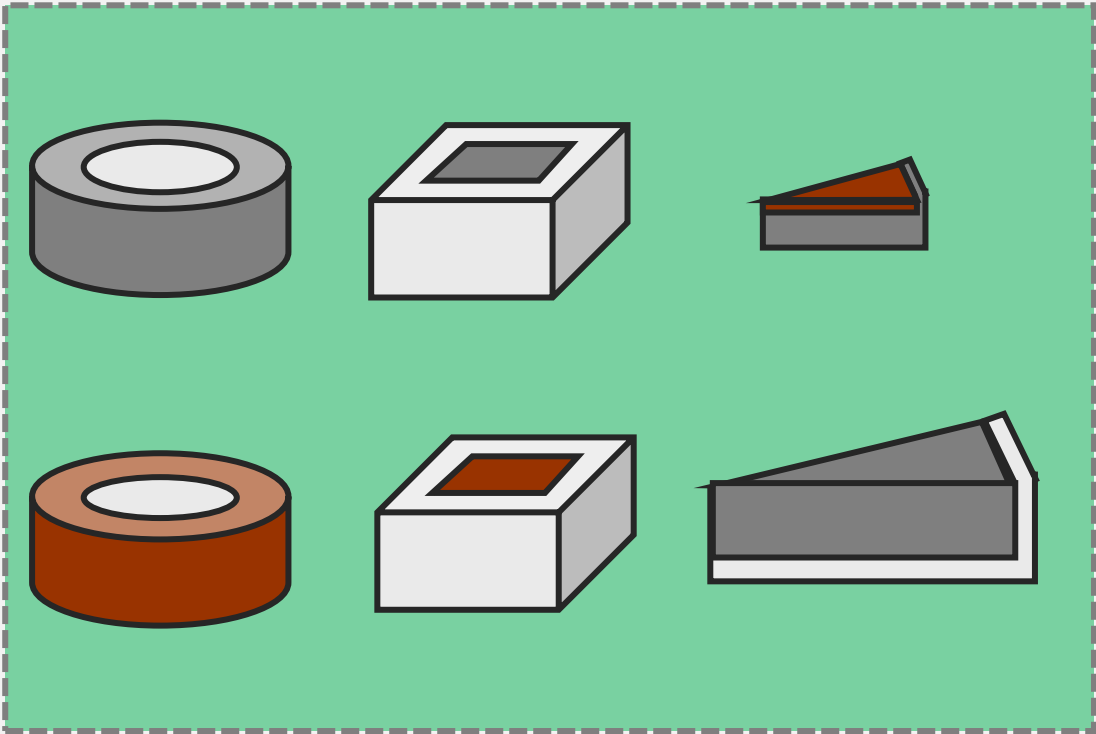


Example	Crust			Filling		Class
	Shape	Size	Shade	Size	Shade	
Ex1	Circle	Large	Grey	Thick	Dark	Pos
Ex2	Circle	Large	White	Thick	Dark	Pos
Ex3	Triangle	Large	Dark	Thick	Grey	Pos
Ex4	Circle	Small	White	Thin	Dark	Pos
Ex5	Square	Large	Dark	Thin	White	Pos
Ex6	Circle	Large	White	Thin	Dark	Pos

Example Source: Kubat, M., 2017. *An introduction to machine learning* (Vol. 2). Cham, Switzerland: Springer International Publishing.

Training data: Negative example

I DO NOT LIKE these types of cake (negatives)



Example	Crust			Filling		Class
	Shape	Size	Shade	Size	Shade	
Ex7	Circle	Large	Grey	Thick	White	Neg
Ex8	Square	Large	White	Thick	Grey	Neg
Ex9	Triangle	Small	Grey	Thin	Dark	Neg
Ex10	Circle	Large	Dark	Thick	White	Neg
Ex11	Square	Large	White	Thick	Dark	Neg
Ex12	Triangle	Large	White	Thick	Grey	Neg

Example Source: Kubat, M., 2017. *An introduction to machine learning* (Vol. 2). Cham, Switzerland: Springer International Publishing.

Training data: All examples

#	Shape	Crust		Filling		Class
		Size	Shade	Size	Shade	
Ex1	Circle	Large	Grey	Thick	Dark	Pos
Ex2	Circle	Large	White	Thick	Dark	Pos
Ex3	Triangle	Large	Dark	Thick	Grey	Pos
Ex4	Circle	Small	White	Thin	Dark	Pos
Ex5	Square	Large	Dark	Thin	White	Pos
Ex6	Circle	Large	White	Thin	Dark	Pos
Ex7	Circle	Large	Grey	Thick	White	Neg
Ex8	Square	Large	White	Thick	Grey	Neg
Ex9	Triangle	Small	Grey	Thin	Dark	Neg
Ex10	Circle	Large	Dark	Thick	White	Neg
Ex11	Square	Large	White	Thick	Dark	Neg
Ex12	Triangle	Large	White	Thick	Grey	Neg

Instance space

All possible combinations

$$|\text{Shape}| \times |\text{Crust}_{\text{size}}| \times |\text{Crust}_{\text{shape}}| \times |\text{Fill}_{\text{size}}| \times |\text{Fill}_{\text{shade}}|$$

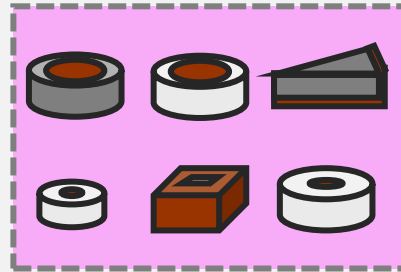
$$3 \times 2 \times 3 \times 2 \times 3$$

108

Example Source: Kubat, M., 2017. *An introduction to machine learning* (Vol. 2). Cham, Switzerland: Springer International Publishing.

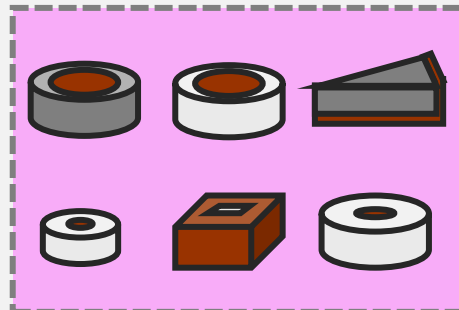
Probability picking a “pos” example (cake that I like) randomly?

I LIKE these types of cake (positives)

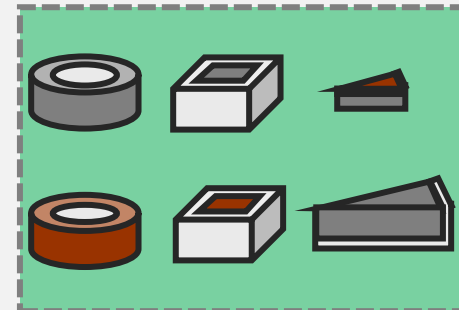


$$P(\text{pos}) = \frac{N_{\text{pos examples}}}{N_{\text{all examples}}} = \frac{6}{12} = 0.5$$

I LIKE these types of cake (positives)



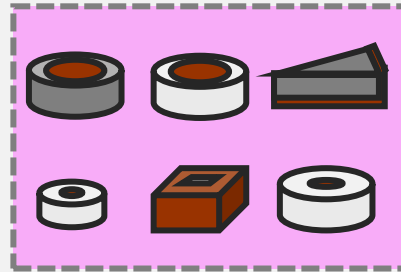
I DO NOT LIKE these types of cake (negatives)



The Prior Probability

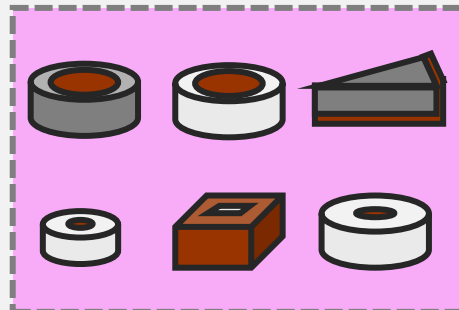
probability of picking a “pos” example randomly?

I LIKE these types of cake (positives)

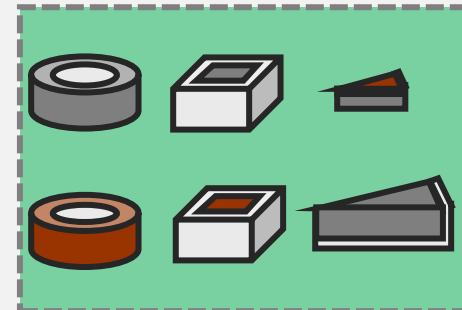


$$P(\text{pos}) = \frac{N_{\text{pos examples}}}{N_{\text{all examples}}} = \frac{6}{12} = 0.5$$

I LIKE these types of cake (positives)



I DO NOT LIKE these types of cake (negatives)



The Conditional Probability:

The probability of picking a “pos” from all “**thick filling**” example randomly?

given

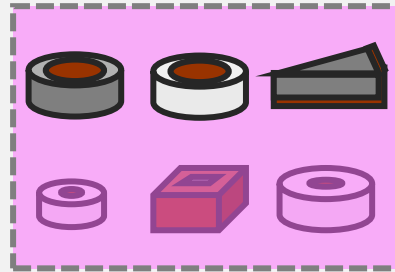
$$P(\text{pos} \mid \text{thick}) = \frac{N_{\text{pos} \mid \text{thick}}}{N_{\text{thick}}} = \frac{3}{8} = 0.375$$



The Conditional Probability

The probability of picking a “thick filling” from all “pos” example randomly?

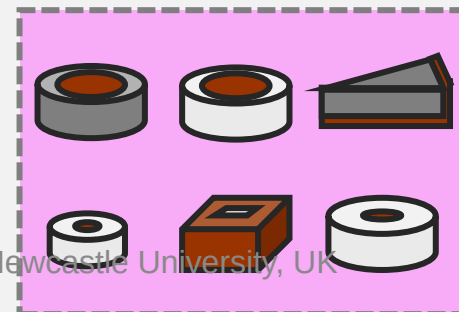
I LIKE these types of cake (positives)



given

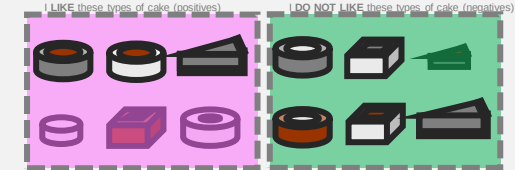
$$P(\text{thick} \mid \text{pos}) = \frac{N_{\text{thick} \mid \text{pos}}}{N_{\text{pos}}} = \frac{3}{6} = 0.5$$

I LIKE these types of cake (positives)

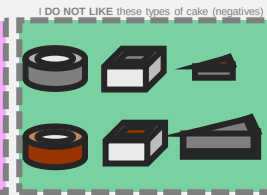
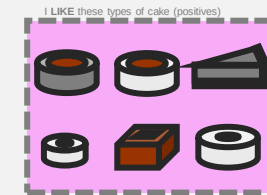
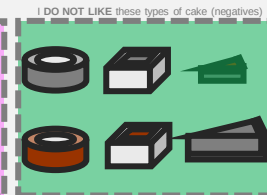
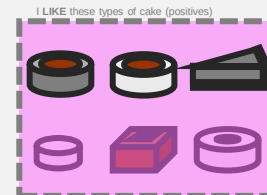


The Joint Probability

The probability of picking a “pos” and “thick filling” example randomly?



$$P(\text{pos}, \text{thick}) = P(\text{pos} \mid \text{thick}) \cdot P(\text{thick})$$



$$= \frac{3}{8} \cdot \frac{8}{12} = \frac{3}{12}$$

The Joint Probability:

The probability of picking a “thick filling” and “pos” example randomly?

$$P(\text{thick}, \text{pos}) = P(\text{thick} \mid \text{pos}) \cdot P(\text{pos})$$

$$= \frac{3}{6} \cdot \frac{6}{12} = \frac{3}{12}$$

The Joint Probability

Two important things

$$P(\text{pos}, \text{thick}) \leq P(\text{pos} \mid \text{thick})$$

Joint probability of two events will always be $\leq$ their conditional probability

less than equal to



$$P(\text{pos}, \text{thick}) = P(\text{thick}, \text{pos})$$

Commutative Property



The Posterior Probability

Probability that I LIKE the cake
given it is of thick filling



$$P(\text{pos} \mid \text{thick}) = \frac{P(\text{thick} \mid \text{pos})P(\text{pos})}{P(\text{thick})}$$

The Posterior Probability:

Bayes Theorem

$$\begin{array}{c} P(\text{pos} \mid \text{thick}) \\ P(\text{Hypothesis} \mid \text{Evidence}) \end{array} = \frac{\begin{array}{c} P(\text{thick} \mid \text{pos})P(\text{pos}) \\ P(\text{Evidence} \mid \text{Hypothesis}) P(\text{Hypothesis}) \end{array}}{\begin{array}{c} P(\text{thick}) \\ P(\text{Evidence}) \end{array}}$$

The Posterior Probability: $P(H|E) = \frac{P(E|H)P(H)}{P(E)}$

The posterior probability of a class given input data

$$P(\text{class} | \text{data}) = \frac{P(\text{data} | \text{class})P(\text{class})}{P(\text{data})}$$

Diagram illustrating the components of the posterior probability formula:

- Top Row:** Evidence features (Circle, Large, Grey, Thick, Dark) are grouped by a vertical line. A label *pos* with a wavy arrow points to the *P(Evidence|Hypothesis)* term in the numerator.
- Bottom Row:** Hypothesis features (Circle, Large, Grey, Thick, Dark) are grouped by a vertical line. A label *pos* with a wavy arrow points to the *P(Hypothesis)* term in the numerator.
- Denominator:** The term *P(Evidence)* is shown below the denominator, with a wavy arrow pointing to the *P(data)* term.

The Posterior Probability:

The posterior probability of a class c_j given an input vector $\mathbf{x}$

The diagram illustrates the posterior probability formula $P(c_j | \mathbf{x}) = \frac{P(\mathbf{x} | c_j) P(c_j)}{P(\mathbf{x})}$. It features several annotations:

- Feature labels (Circle, Large, Grey, Thick, Dark) are placed above the terms $\mathbf{x}$ in both the numerator and denominator.
- Class labels (c_j) are highlighted in pink.
- Blue arrows point from the class labels to the words "pos" (positive) and "neg" (negative), indicating their possible values.

$$P(c_j | \mathbf{x}) = \frac{P(\mathbf{x} | c_j) P(c_j)}{P(\mathbf{x})}$$

Where $\mathbf{x} = \langle x_1, x_2, \dots, x_n \rangle$ is a data and $c_j = f(\mathbf{x})$, class label, e.g., $c_1 = pos$ and $c_2 = neg$

In blue text the vector $\mathbf{x}$ and class c_j is only given as an example, this could be any value from the table in slide 37

The Posterior Probability:

The posterior probability of a class c_j given an input vector $\mathbf{x}$

$$P(c_j | \mathbf{x}) = P(\mathbf{x} | c_j)P(c_j)$$

Where $\mathbf{x} = \langle x_1, x_2, \dots, x_n \rangle$ is a data and $c_i = f(\mathbf{x})$, class label

Since $P(\mathbf{x})$ being same for all classes in question, we will label data with class which maximises the numerator $P(\mathbf{x} | c_j)P(c_j)$

The Prior Probability $P(c_j)$ is easy!

$$P(c_j) = \frac{N_{\text{examples of class labeled } c_j}}{N_{\text{all examples}}}$$

The Prior Probability $P(\mathbf{x} | c_j)$ is hard!

$$P(\mathbf{x} | c_j) = \frac{N_{\text{examples represent vector } \mathbf{x} \text{ where class is } c_j}}{N_{\text{all examples labeled as class } c_j}}$$

The Prior Probability $P(\mathbf{x} | c_j)$ is hard!

Instance space can be huge!

What if the vector $\mathbf{x}$ does not belong to the training set?

$$P(\mathbf{x} | c_j) = \frac{N_{\text{examples represent vector } \mathbf{x} \text{ where class is } c_j}}{N_{\text{all examples labeled as class } c_j}}$$

$\mathbf{x} =$ | Triangle Small Grey Thin Grey |

 This example does **NOT** belong to the training data

The Prior Probability $P(\mathbf{x} | c_j)$ is hard!

Instance space can be huge!

What if the vector $\mathbf{x}$ does not belong to the training set?

$$P(c_j | \mathbf{x}) = P(\mathbf{x} | c_j) P(c_j) = 0 \cdot P(c_j) = \mathbf{0}$$

The formula will give **0** **That's bad for classifier!!**

Hope! Try individual attributes

Assumption! Mutually independent attributes:

Calculate each attribute's probability separately (independently)

The diagram shows the formula $P(\mathbf{x} | c_j) = \prod_{i=1}^n P(x_i | c_j)$ with several annotations:

- An arrow labeled "limit" points to the upper index n of the product.
- An arrow labeled "product (multiplication) for $i = 1$ to n multiply $p(.|.)$ " points to the product symbol $\prod$.
- An arrow labeled "iterator (index)" points to the lower index $i=1$.
- An arrow points to the term $P(x_i | c_j)$ with the text "e.g., x_i (circle) given C_j (pos) This we can easily find in the training data".

Where $\mathbf{x} = \langle x_1, x_2, \dots, x_n \rangle$ is a data, n is number of features (columns) and $c_j = f(\mathbf{x})$ class label

The Posterior Probability:

The posterior probability of a class given input data vector

$$P(c_j | \mathbf{x}) = P(c_j) \cdot \prod_{i=1}^n P(x_i | c_j)$$

Since $P(\mathbf{x})$ being same for all classes in question, we will label data $(\mathbf{x})$ with class (c_j) which maximises the numerator $P(\mathbf{x} | c_j)P(c_j)$.

Because we make a naïve assumption

Naïve Bayes Classifier (NBC):

Maximum a Posteriori (MAP)

Returns a number
between 1 to k for which
value is maximum

$$\hat{y} = \underset{c_j \in \{1, 2, \dots, k\}}{\operatorname{argmax}} P(c_j) \cdot \prod_{i=1}^n P(x_i | c_j)$$

Because we make a naïve assumption

Naïve Bayes Classifier (NBC):

Maximum a Posteriori (MAP) using **log likelihood**

$$\hat{y} = \operatorname{argmax}_{c_j \in \{1, 2, \dots, k\}} \log \left(P(c_j) \right) \sum_{i=1}^n \log \left(P(x_i | c_j) \right)$$

Sum (addition)

The log-likelihood in Naive Bayes (NBC) is beneficial because it converts the product of many small probabilities into a sum, which is numerically more stable and computationally faster. It avoids numerical underflow, allows for easier comparison between classes, and simplifies calculations by turning multiplication into addition.

Naïve Bayes Classifier: Balanced Dataset

In training data, each class has equal distribution

Assuming a **uniform prior** $P(c_j)$ over the hypothesis space, MAP reduces to Maximum Likelihood learning:

$$\hat{y} = \operatorname{argmax}_{c_j \in \{1, 2, \dots, k\}} \prod_{i=1}^n P(x_i | c_j)$$

Maximum Likelihood learning

In training data, each class has equal distribution

Assuming a **uniform prior** $P(c_j)$ over the hypothesis space, MAP reduces to Maximum Likelihood learning:

$$\hat{y} = \operatorname{argmax}_{c_j \in \{1, 2, \dots, k\}} \prod_{i=1}^n P(x_i | c_j)$$

Bayesian Classifier

Continuous domain

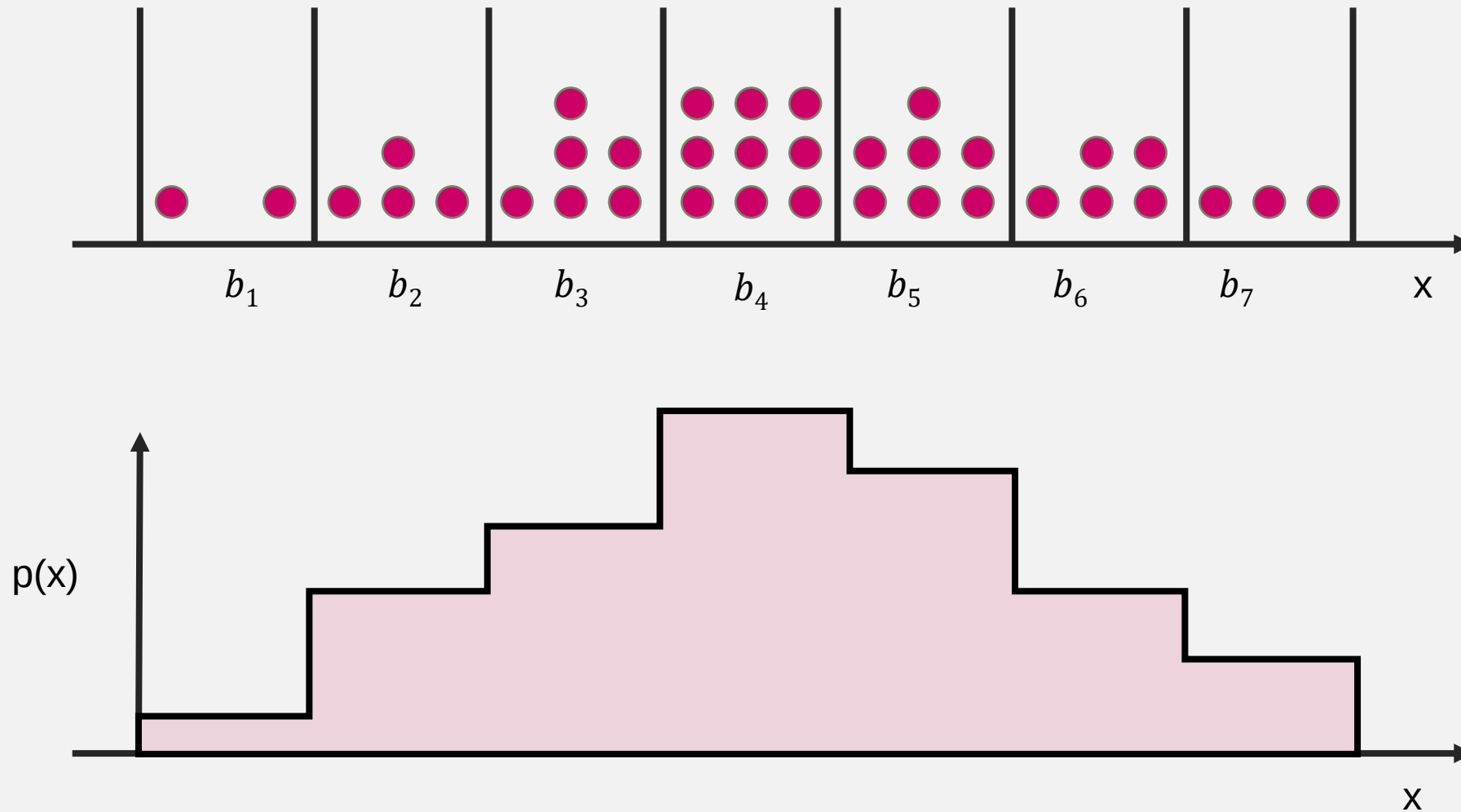


Does NBC also work for continuous attributes?

If we try to find frequency of mutually independent attribute x_i which is a *real number* (e.g., *weight of a person 60.015 kg*) and not discrete (e.g., Adult, Child), the Prior Probability $P(x_i | c_j)$ is super hard to find in a continuous domain.

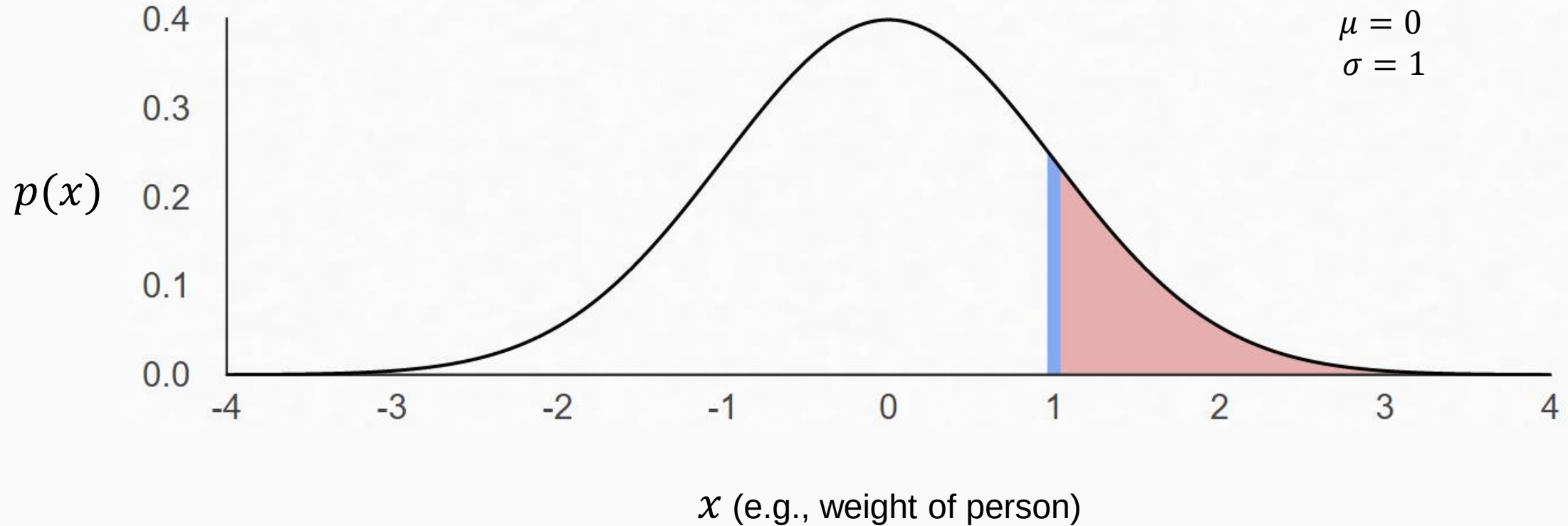
Instance-space is too vast!

Binning of Continues Variables



Probability Density Function (pdf)

Gaussian function



Probability Density Function (pdf)

Gaussian function

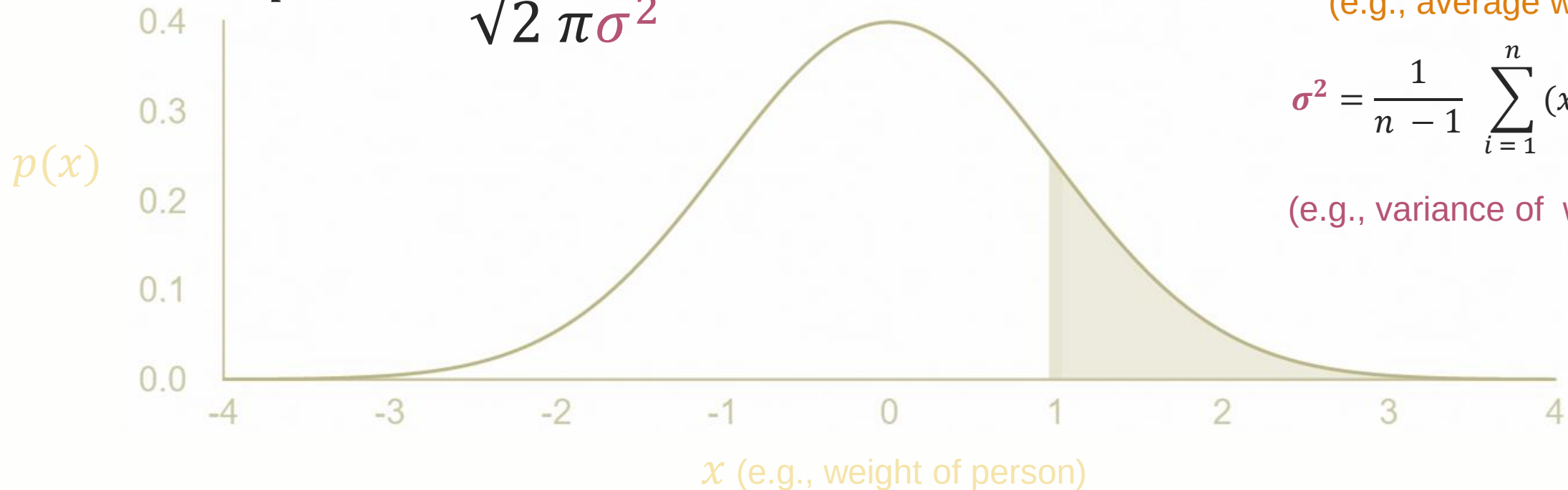
$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \cdot e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

$$\mu = \frac{1}{n} \sum_{i=1}^n x_i$$

(e.g., average weight)

$$\sigma^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \mu)^2$$

(e.g., variance of weight)



The Posterior Probability:

The posterior probability of a class given input (continuous) variable x

$$P(c_j | x_i) = \frac{p_{c_j}(x_i) \cdot P(c_j)}{p(x_i)}$$

$p(x)$ is a probability density function over variable x and $c_j = f(x)$ is a class label

Naïve Bayes Classifier (Continuous Variable)

In training data, each class has equal distribution

Assuming a **uniform prior** $P(c_j)$ over the hypothesis space, MAP reduces to Maximum Likelihood learning:

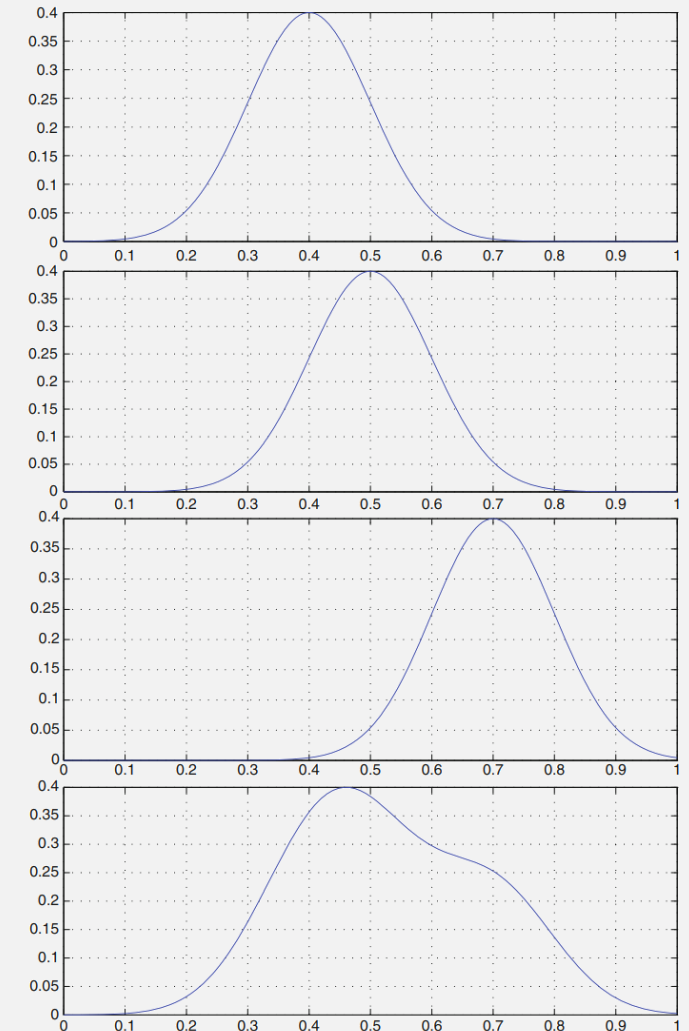
$$\hat{y} = \operatorname{argmax}_{c_j \in \{1, 2, \dots, k\}} \prod_{i=1}^n p_{c_j}(x_i | c_j)$$

Combining Gaussian function (pdfs)

If we denote by x_k (as being mean μ_k) the value of x in the k -th example, then the pdf is approximated by the following **sum of m functions**:

$$p_{c_j}(x_i) = \frac{1}{m \sqrt{2 \pi \sigma^2}} \sum_{k=1}^m e^{-\frac{(x_i - x_k)^2}{2 \sigma^2}}$$

x_i is the test input, m being total number of examples in a training set labelled as c_j



Example Table

Set	Examples	Attributes			Class
		A	B	C	
Training	Ex1	3.2	2.1	2.1	Pos
	Ex2	5.2	6.1	7.5	Pos
	Ex3	8.5	1.3	0.5	Pos
	Ex4	2.3	5.4	2.45	Neg
	Ex5	6.2	3.1	4.4	Neg
	Ex6	1.3	6.0	3.35	Neg
Test	Ex7	9.0	3.6	3.3	Pos / Neg ?

Try yourself

For the given training example (Table in Slide #62), compute the following, we use $\sigma^2 = 1$:

$$\text{If } p_{\text{pos}}(A_{ex_7}) = \frac{1}{3\sqrt{2\pi}} \left[e^{-0.5(A_{ex_7}-A_{ex_1})^2} + e^{-0.5(A_{ex_7}-A_{ex_2})^2} + e^{-0.5(A_{ex_7}-A_{ex_3})^2} \right]$$

$$\text{compute } p_{\text{pos}}(\mathbf{x}_7) = p_{\text{pos}}(A_{ex_7}) \cdot p_{\text{pos}}(B_{ex_7}) \cdot p_{\text{pos}}(C_{ex_7})$$

$$\text{If } p_{\text{neg}}(A_{ex_7}) = \frac{1}{3\sqrt{2\pi}} \left[e^{-0.5(A_{ex_7}-A_{ex_4})^2} + e^{-0.5(A_{ex_7}-A_{ex_5})^2} + e^{-0.5(A_{ex_7}-A_{ex_6})^2} \right]$$

$$\text{compute } p_{\text{neg}}(\mathbf{x}_7) = p_{\text{neg}}(A_{ex_7}) \cdot p_{\text{neg}}(B_{ex_7}) \cdot p_{\text{neg}}(C_{ex_7})$$

determine the output class by computing $\hat{y} = \text{argmax}(p_{\text{pos}}(\mathbf{x}_7), p_{\text{neg}}(\mathbf{x}_7))$

Part 2/c

Tree-Based Classifier

PROBLEMS IN THE ANALYSIS OF SURVEY DATA, AND A PROPOSAL

JAMES N. MORGAN AND JOHN A. SONQUIST*

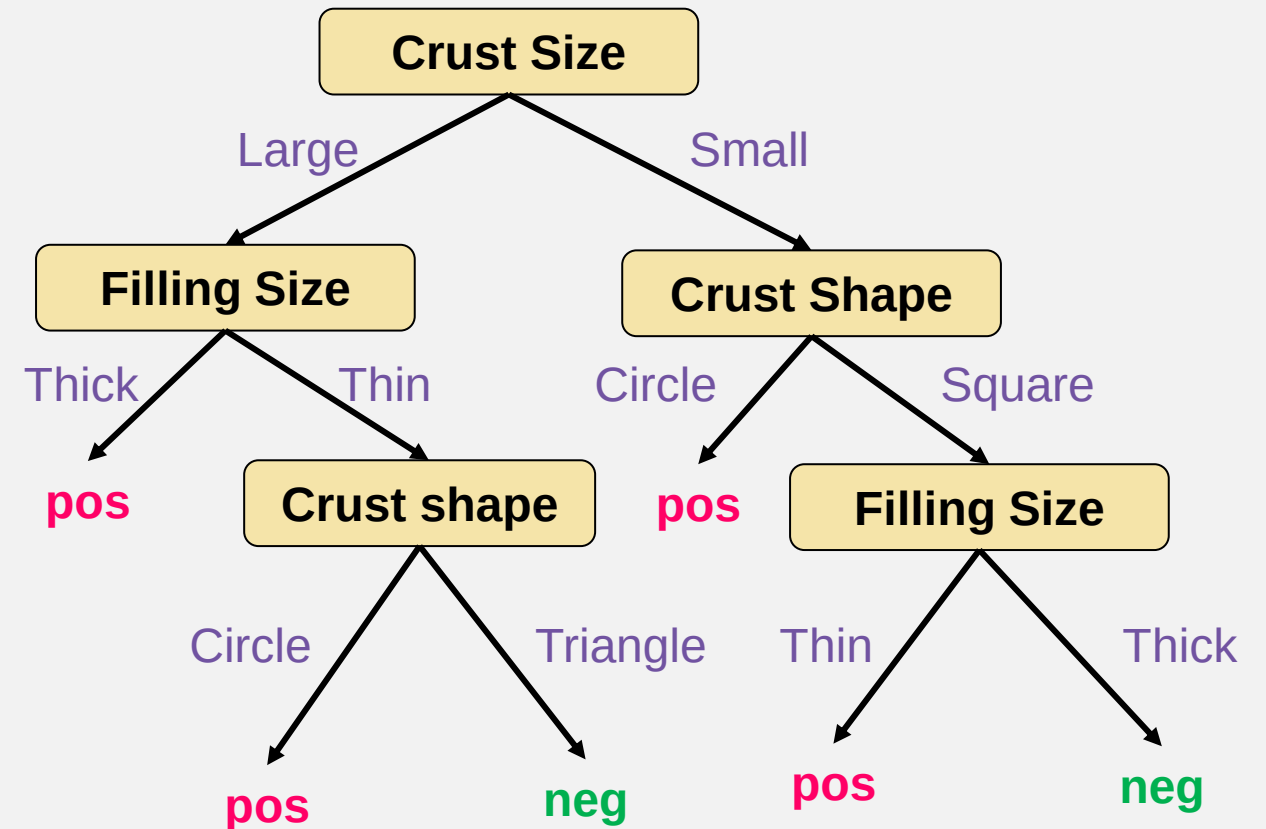
University of Michigan

Most of the problems of analyzing survey data have been reasonably well handled, except those revolving around the existence of interaction effects. Indeed, increased efficiency in handling multivariate analyses even with non-numerical variables, has been achieved largely by assuming additivity. An approach to survey data is proposed which imposes no restrictions on interaction effects, focuses on importance in reducing predictive error, operates sequentially, and is independent of the extent of linearity in the classifications or the order in which the explanatory factors are introduced.

A. NATURE OF THE DATA AND THE WORLD FROM WHICH THEY COME

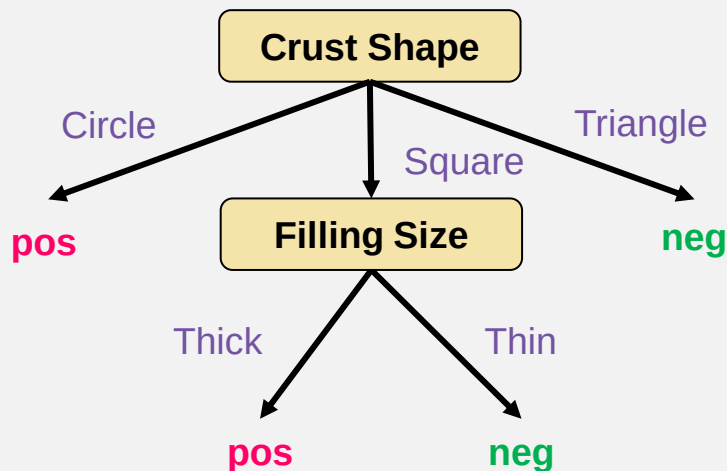
Decision Tree

#	Crust		Filling	Class
	Shape	Size	Size	
Ex1	Circle	Large	Thin	Pos
Ex2	Circle	Small	Thin	Pos
Ex3	Square	Large	Thin	Neg
Ex4	triangle	Large	Thin	Neg
Ex5	Square	Large	Thick	Pos
Ex6	Square	Small	Thin	Neg
Ex7	Square	Small	Thick	Pos
Ex8	Circle	Large	Thick	Pos



Decision Tree

#	Crust		Filling	Class
	Shape	Size	Size	
Ex1	Circle	Large	Thin	Pos
Ex2	Circle	Small	Thin	Pos
Ex3	Square	Large	Thin	Neg
Ex4	triangle	Large	Thin	Neg
Ex5	Square	Large	Thick	Pos
Ex6	Square	Small	Thin	Neg
Ex7	Square	Small	Thick	Pos
Ex8	Circle	Large	Thick	Pos



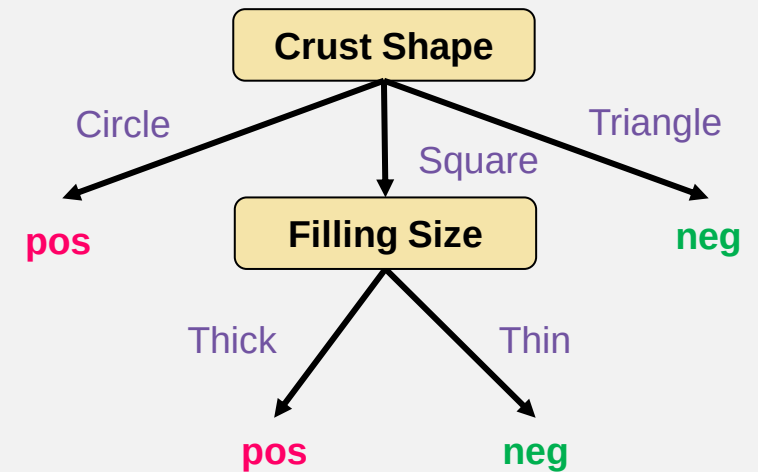
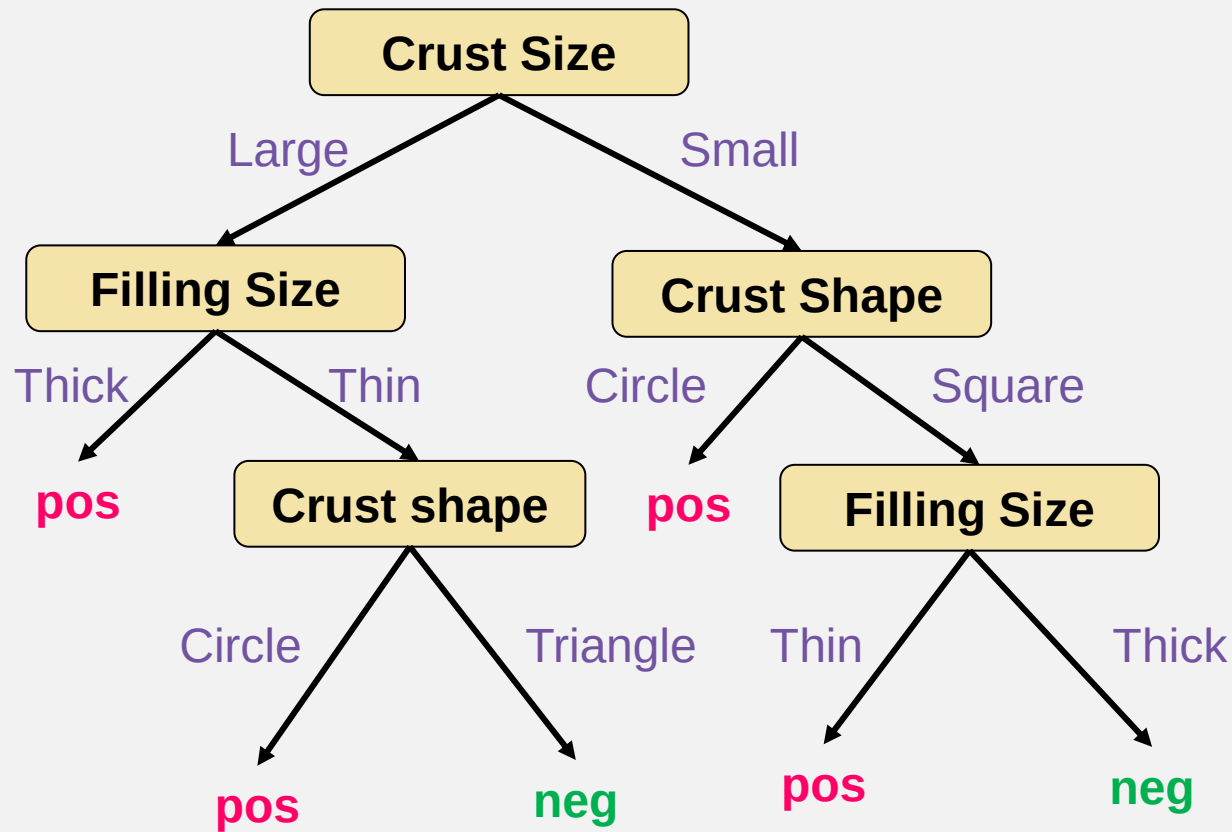
For training data T

$\text{growTree}(T)$:

1. Find an attribute $a \in A$, (e.g. $A = \{\text{Shape, Size, Filling size}\}$) that **maximize the information** about the class labels
2. Divide T into subsets T_i , each characterised by different values of $v \in \text{values}(a)$
3. For each T_i :
if all example in T_i belongs to same class then create leaf labelled with that class, else recursively $\text{growTree}(T_i)$

Which tree is better?

Depending on which attribute and their values you chose at the start there can be different trees



Entropy

Thermodynamics: degree of disorder or randomness in the system

Information Theory: a measure of the uncertainty or average information content of a random variable



Shannons Entropy

Entropy (H)

Information Theory: a measure of the uncertainty or average information content of a random variable

$$H(T) = - \sum_{c \in \mathcal{C}} p(c) \log_2 p(c)$$

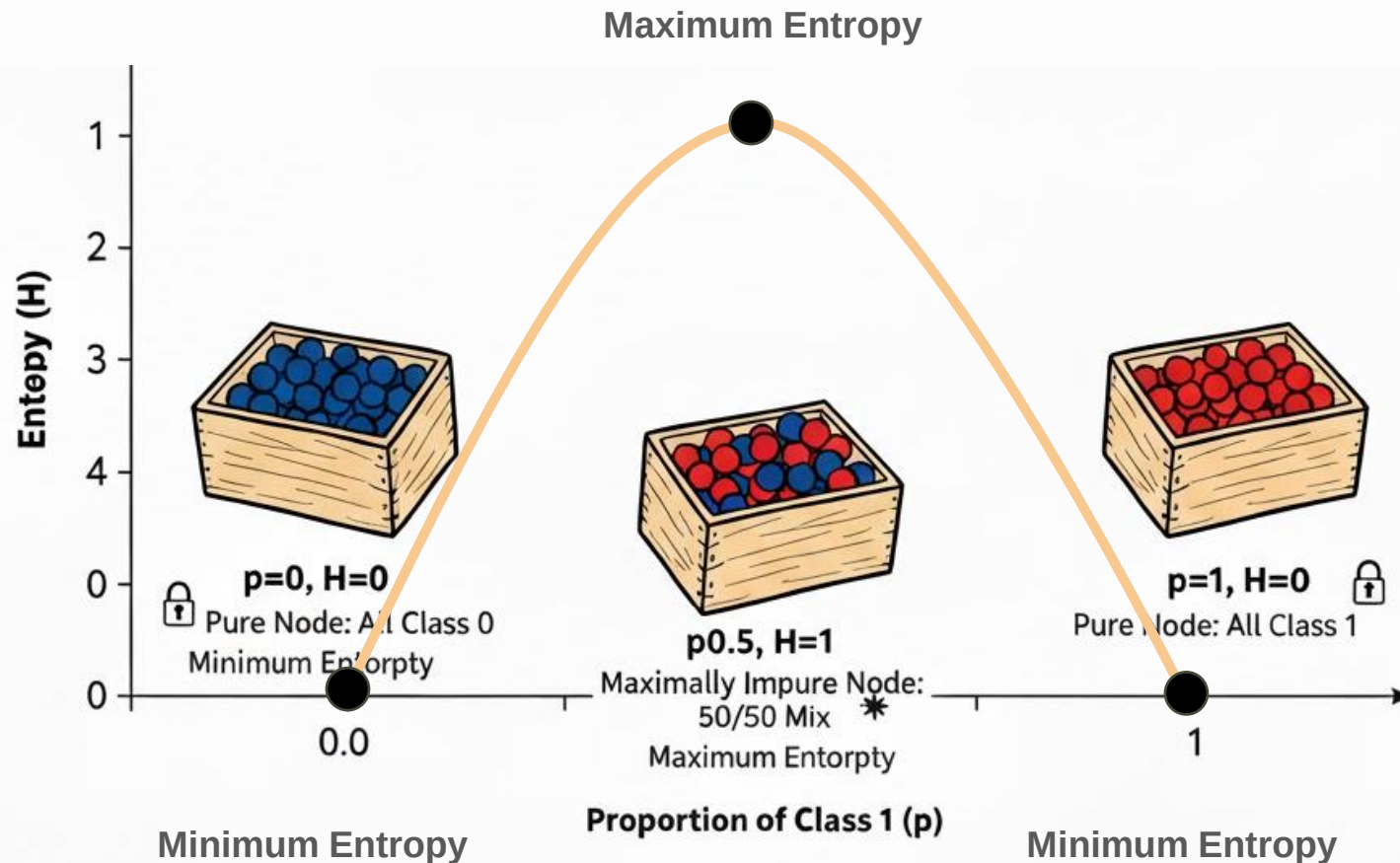
belongs to



Where T is dataset (training set) and $\mathcal{C}$ is a set of classes e.g., $\mathcal{C} = \{pos, neg\}$ and $p(c)$ probability of a class c in T . Entropy $H(T)$ is also the measure of impurity of training set T .

Entropy (H)

Information Theory: a measure of the uncertainty or average information content of a random variable



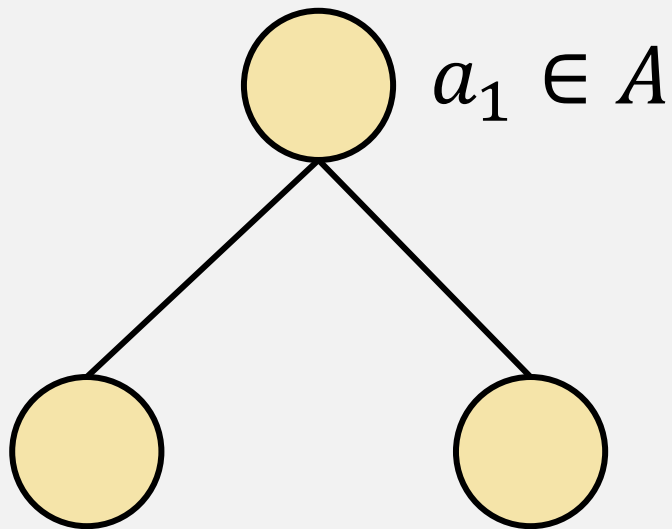
$H(T) = 0$ (Minimum Entropy Pure Node): When the proportion of Class is 0 or 1, the node is pure (all samples belong to one class). There is no uncertainty about the class of a randomly chosen sample.

$H(T) = 1$ (Maximum Impurity/Uncertainty/Degree of surprise): When the proportion of Class 1 is 0.5 (an equal split of classes), the node is maximally impure. There's maximum uncertainty about the class of a randomly chosen sample.

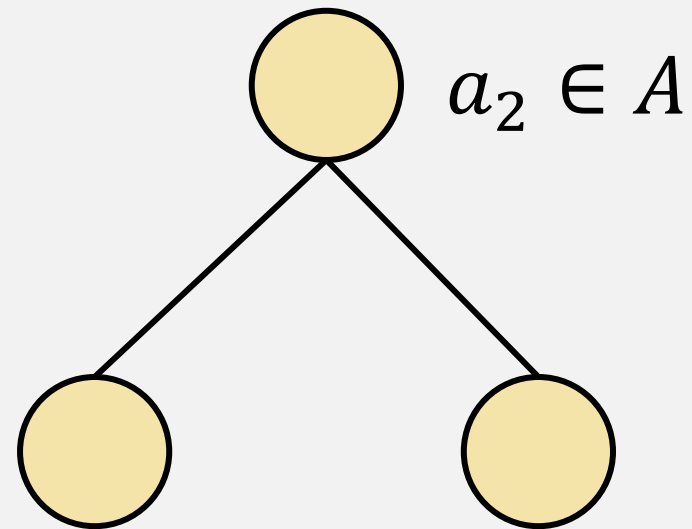
Information gain due to split

Which split is better?

Split 1



Split 2



Information Gain (I)

Information gain refers to the measure of entropy after a decision tree node split due to an attribute $a \in A$ of training data T . It indicate **reduction in entropy** on sorting T after split on $a \in A$

$$I(T, a) = H(T) - H(T|a)$$

$$H(T|a) = \sum_{i \in \text{values}(a)} \frac{|T_i|}{|T|} H(T_i)$$

Where T_i is the number remaining examples in training set after the split and $|T|$ total number of examples in dataset T

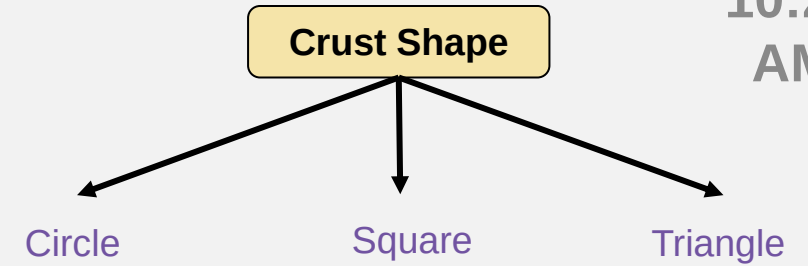
Information Maximization Algorithm

- Calculate the entropy of training data T

$$H(T) = - \sum_{c \in \mathcal{C}} p(c) \log_2 p(c)$$

- For each attribute $a \in A$ of training data T , divide T into subset T_i and calculate the following:
 - Calculate entropy of each subset T_i : $H(T_i)$
 - Calculate average entropy at attribute a : $H(T|a) = \sum_{i \in \text{values}(a)} \frac{|T_i|}{|T|} H(T_i)$
 - Calculate information gain $I(T, a) = H(T) - H(T|a)$
- Chose the attribute with highest information gain for the node split and recurse $\text{growTree}(T_i)$

Try yourself

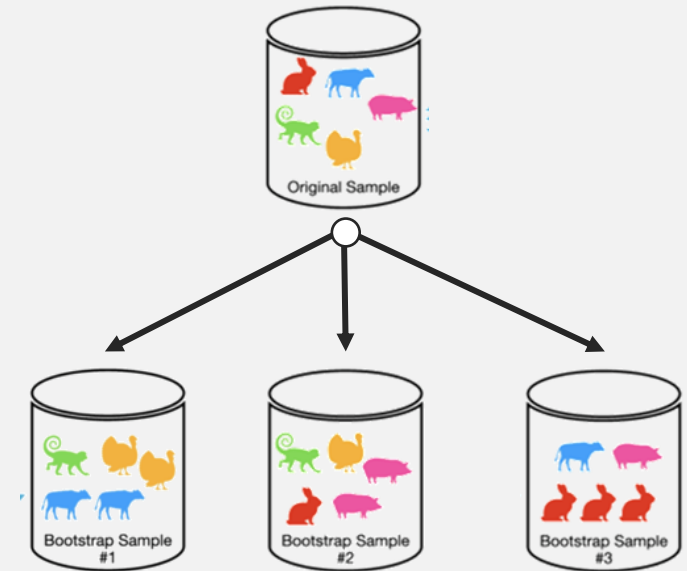


- Calculate entropy $H(T)$ of the training set
 - Answer: 0.954
- Calculate entropy $H(T_i | a)$ of the subset T_i for crust shape, crust size and filling size
 - Answer: 0.5, 0.951, 0.607
- Calculate the information gain for each attribute crust shape, crust size and filling size
 - Answer: 0.454, 0.003, 0.347

Ensemble learners: Several weak classifiers are better than one strong classifier

Bagging (bootstrap aggregation)

- Create n equal number of training subset from original training set T using sampling with replacement (by bootstrapping)
- For each subset create a decision tree or any other classifier, therefore we get n decision trees from n subset
- A master classifier then use voting (e.g., majority voting or similar method) to decide classification results (by aggregation)





Random Forests

LEO BREIMAN

Statistics Department, University of California, Berkeley, CA 94720

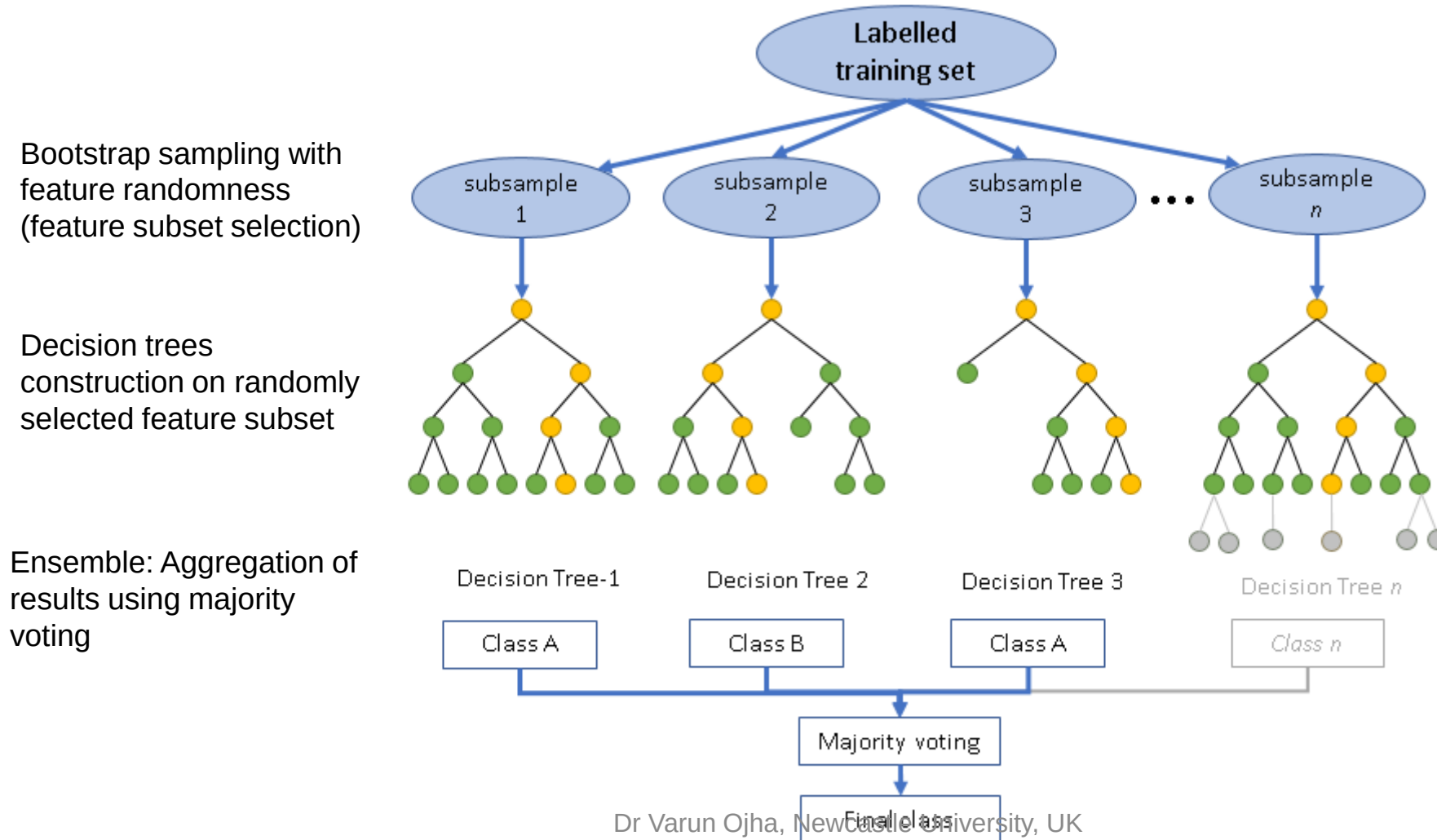
Editor: Robert E. Schapire

Abstract. Random forests are a combination of tree predictors such that each tree depends on the values of a random vector sampled independently and with the same distribution for all trees in the forest. The generalization error for forests converges a.s. to a limit as the number of trees in the forest becomes large. The generalization error of a forest of tree classifiers depends on the strength of the individual trees in the forest and the correlation between them. Using a random selection of features to split each node yields error rates that compare favorably to Adaboost (Y. Freund & R. Schapire, *Machine Learning: Proceedings of the Thirteenth International conference*, * * *, 148–156), but are more robust with respect to noise. Internal estimates monitor error, strength, and correlation and these are used to show the response to increasing the number of features used in the splitting. Internal estimates are also used to measure variable importance. These ideas are also applicable to regression.

Random forest

10:29
AM

Feature randomness, also known as feature bagging generates a random subset of features, which ensures low correlation among decision trees. This is a key difference between decision trees and random forests. While decision trees consider all the possible feature splits, random forests only select a subset of those features



Support-Vector Networks

CORINNA CORTES

VLADIMIR VAPNIK

AT&T Bell Labs., Holmdel, NJ 07733, USA

corinna@neural.att.com

vlad@neural.att.com

Editor: Lorenza Saitta

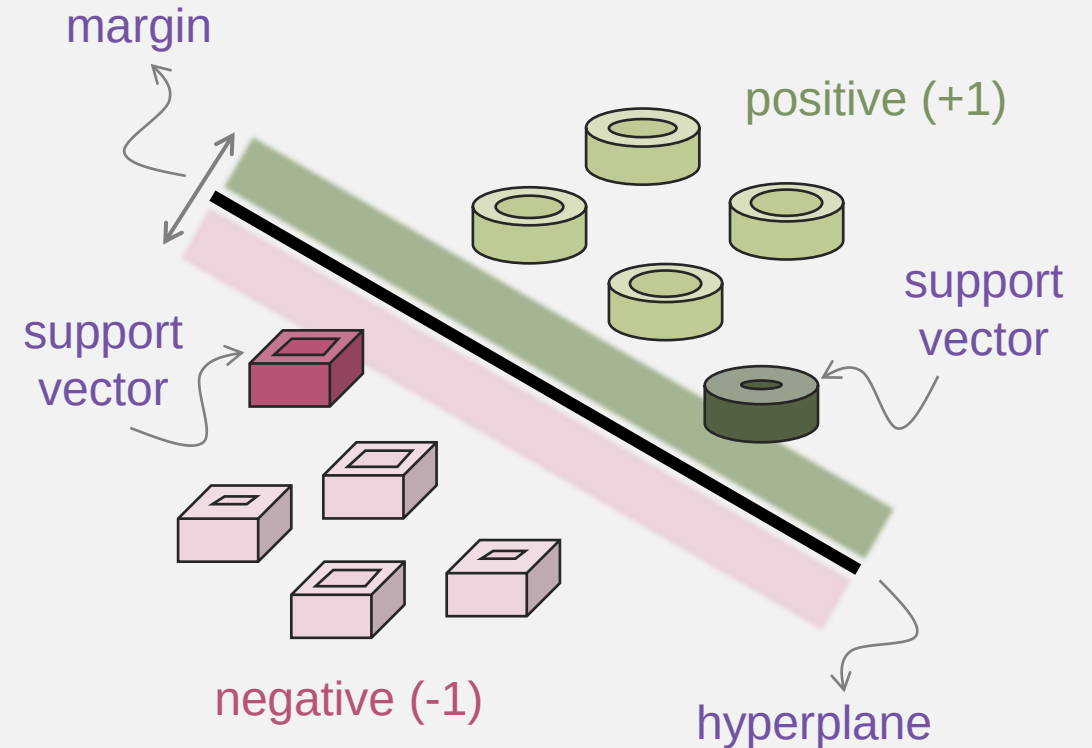
Abstract. The *support-vector network* is a new learning machine for two-group classification problems. The machine conceptually implements the following idea: input vectors are non-linearly mapped to a very high-dimension feature space. In this feature space a linear decision surface is constructed. Special properties of the decision surface ensures high generalization ability of the learning machine. The idea behind the support-vector network was previously implemented for the restricted case where the training data can be separated without errors. We here extend this result to non-separable training data.

High generalization ability of support-vector networks utilizing polynomial input transformations is demonstrated. We also compare the performance of the support-vector network to various classical learning algorithms that all took part in a benchmark study of Optical Character Recognition.

Original idea was first proposed by Vladimir Vapnik and Alexey Chervonenkis in 1964

Support Vector Machine (SVM)

Given a training sample, the support vector machine (SVM) constructs a hyperplane as the decision surface in such a way that the margin of separation between positive and negative examples is maximized.



Support Vector Machine (SVM)

Let the training sample

$$\{(\mathbf{x}_i, d_i)\}_{i=1}^N$$

$\mathbf{x}_i = \{x_1, x_2\}$ is the input

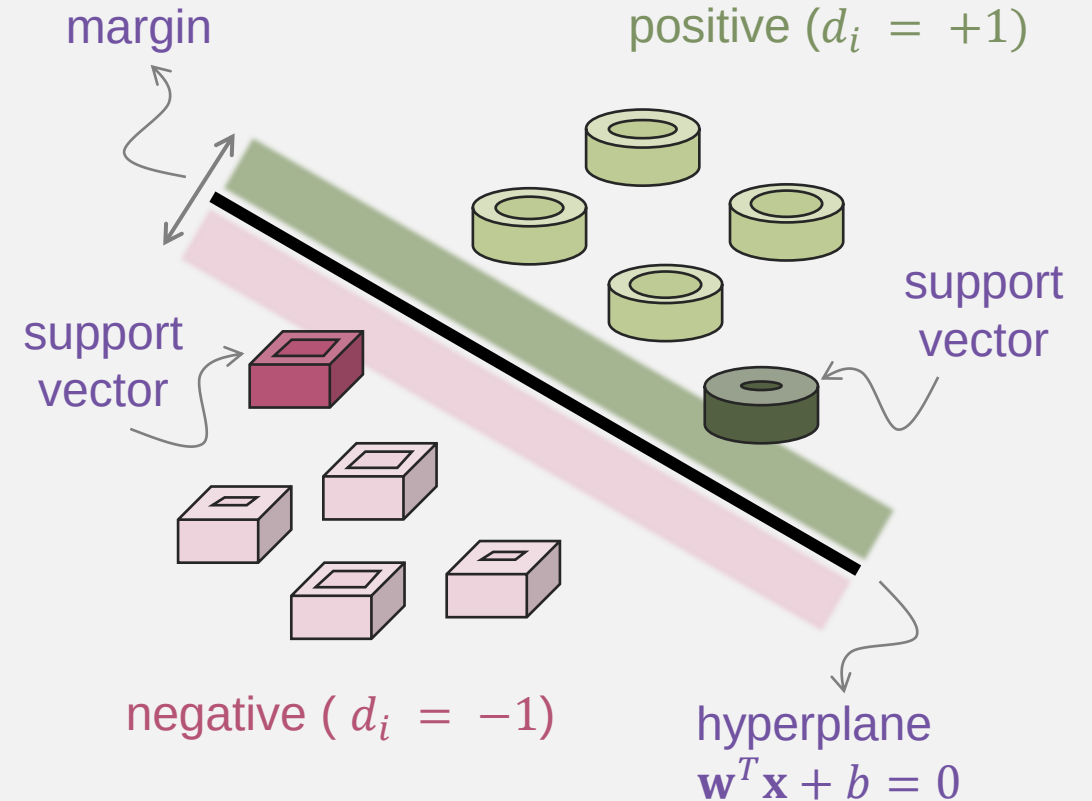
d_i is the corresponding target output/class).

Assumption in SVM. The positive ($d_i = +1$) and negative ($d_i = -1$) classes are “linearly separable.”

The **hyperplane** that separated these two classes is

$$\mathbf{w}^T \mathbf{x} + b = 0$$

w is weight associated with input and b is bias.



Support Vector Machine (SVM)

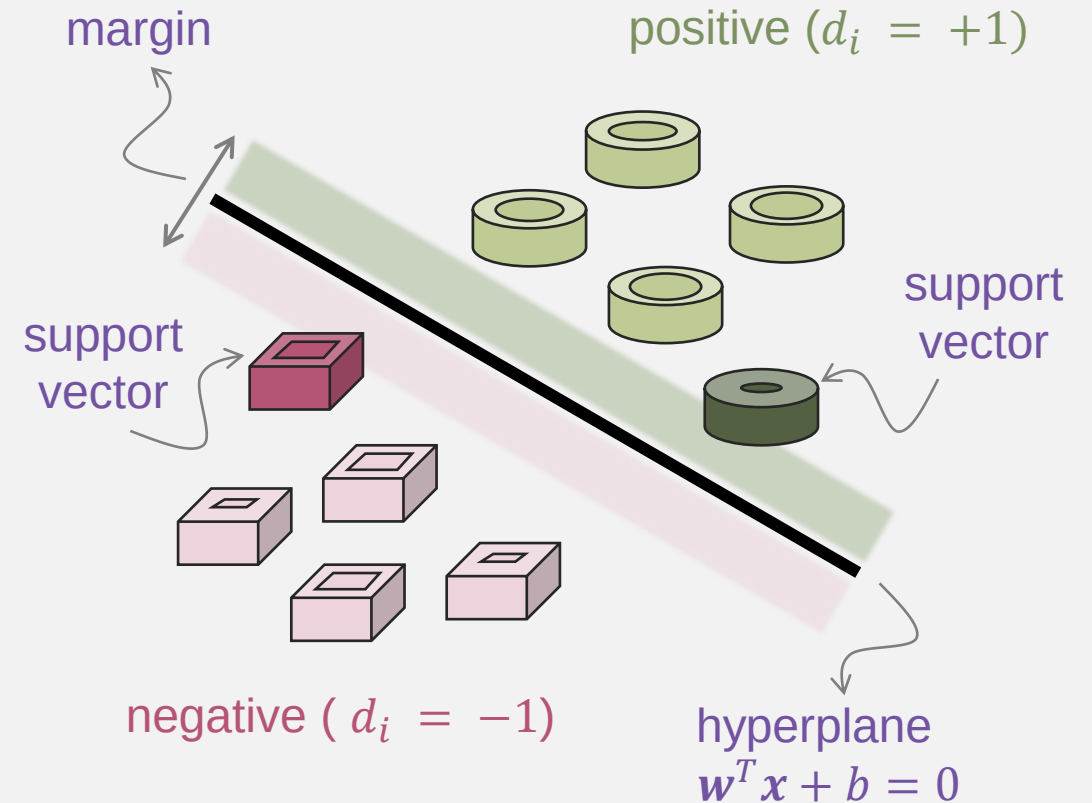
The hyperplane that separated these two classes is

$$\mathbf{w}^T \mathbf{x} + b = 0$$

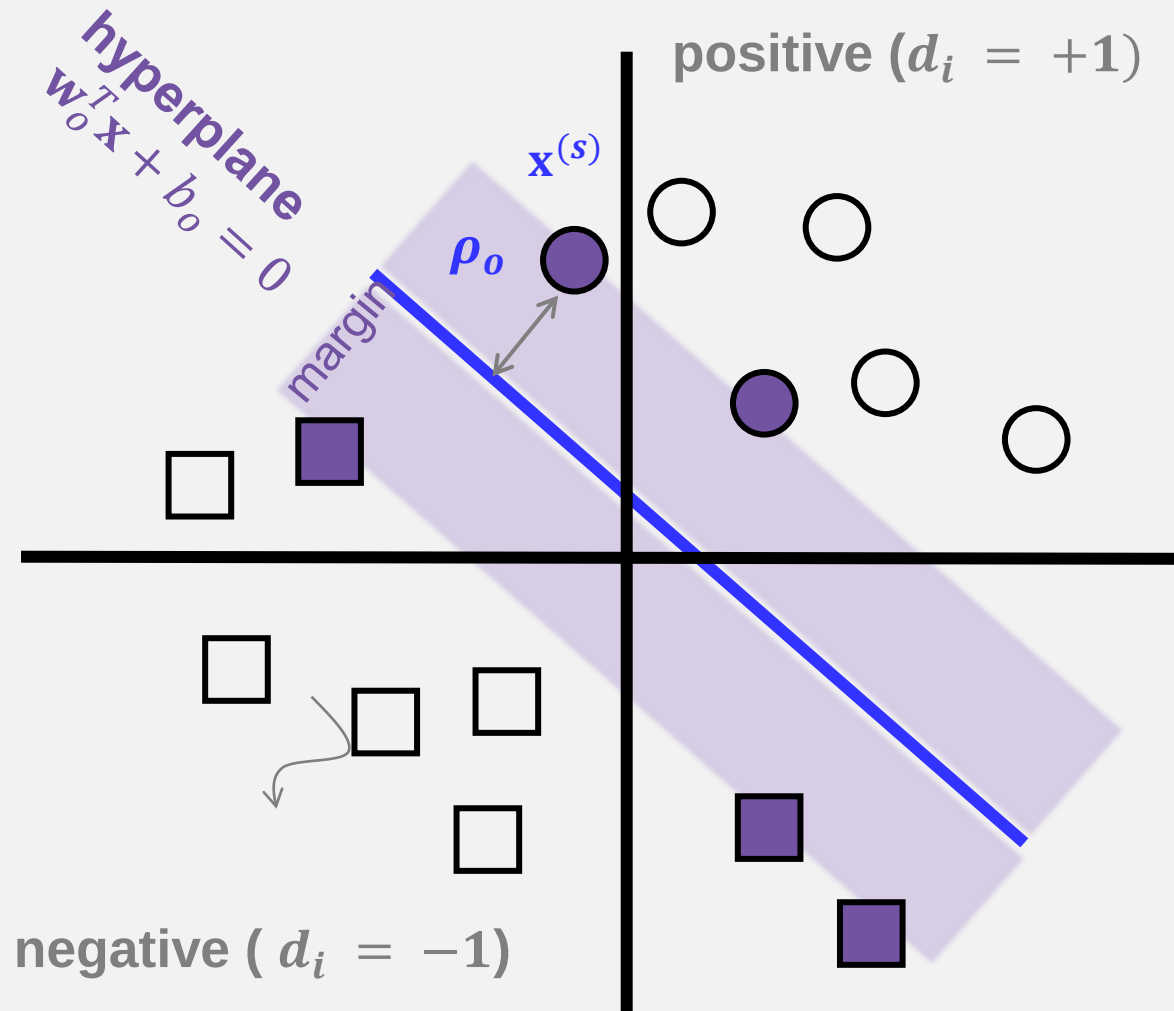
$\mathbf{w}$ is weight associated with input and b is bias.

$$\mathbf{w}^T \mathbf{x} + b \geq 0 \text{ for } d_i = +1$$

$$\mathbf{w}^T \mathbf{x} + b < 0 \text{ for } d_i = -1$$



Support Vector Machine (SVM)



Let the hyperplane has w_o has optimal weight and optimal b_o is bias to separated these two classes with optimal margin of separation ρ_o ,

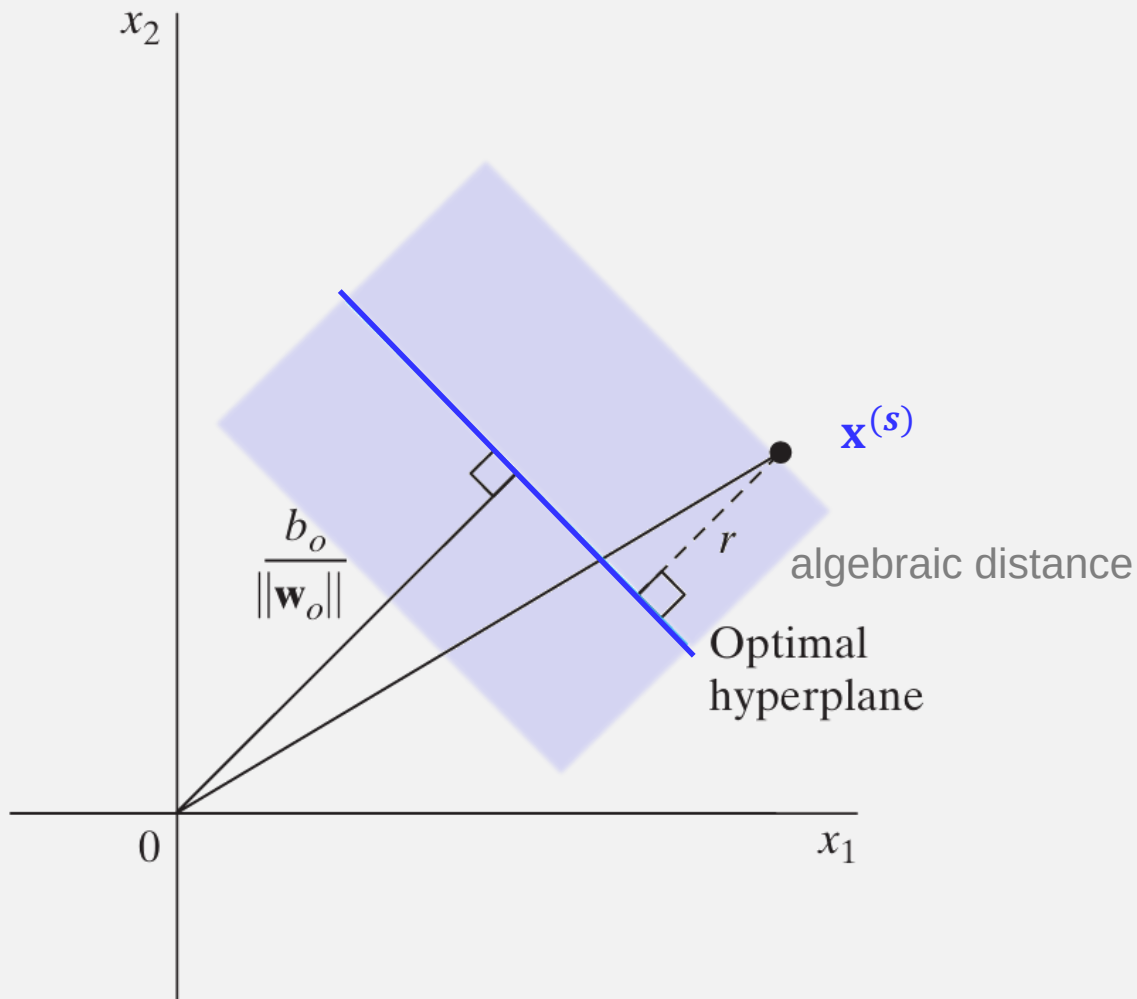
$$w_o^T x + b_o = 0$$

We can write this as discriminant function

$$g(x) = w_o^T x + b_o$$

Support Vector Machine (SVM)

Geometric interpretation of algebraic distances of points to the optimal hyperplane for a 2D case



For support vector $\mathbf{x}^{(s)}$ we write discriminant function as

$$g(\mathbf{x}^{(s)}) = \mathbf{w}_o^T \mathbf{x}^{(s)} + b_o = \pm 1$$

for $d^{(s)} = \pm 1$

The algebraic distance from support vector $\mathbf{x}^{(s)}$ to optimal hyperplane is

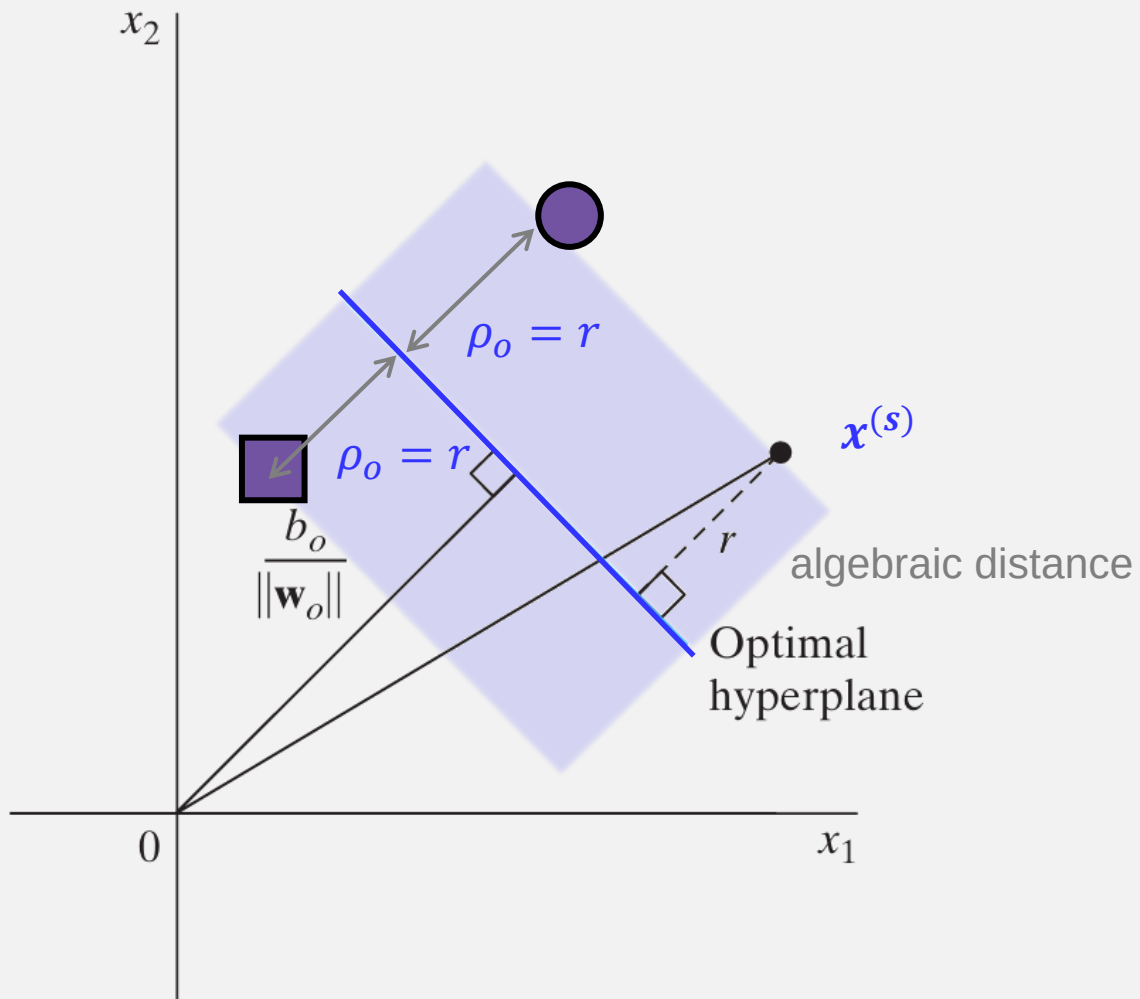
$$g(\mathbf{x}^{(s)}) = \mathbf{w}_o^T \mathbf{x}^{(s)} + b_o = r \|\mathbf{w}_o\|$$

$$r = \frac{g(\mathbf{x}^{(s)})}{\|\mathbf{w}_o\|} \text{ algebraic distance}$$

and

$$r = \frac{1}{\|\mathbf{w}_o\|} \text{ for } d^{(s)} = +1 \text{ and } r = -\frac{1}{\|\mathbf{w}_o\|} \text{ for } d^{(s)} = -1$$

Support Vector Machine (SVM)



Optimal value of the margin of separation between two classes. Since

$$\rho = 2r \text{ (see diagram)}$$

thus

$$\rho = \frac{2}{\|\mathbf{w}_o\|}$$

Thus, maximizing the margin of separation between binary classes is equivalent to minimizing the Euclidean norm of the weight vector $\mathbf{w}$. That is, we have optimization problem of the form

$$\underset{\mathbf{w}, b}{\text{minimize}} \frac{1}{2} \|\mathbf{w}_o\|^2 \text{ alternatively } \underset{\mathbf{w}, b}{\text{minimize}} \frac{1}{2} \mathbf{w}^2 \mathbf{w}$$

Support Vector Machine (SVM)

Algorithm

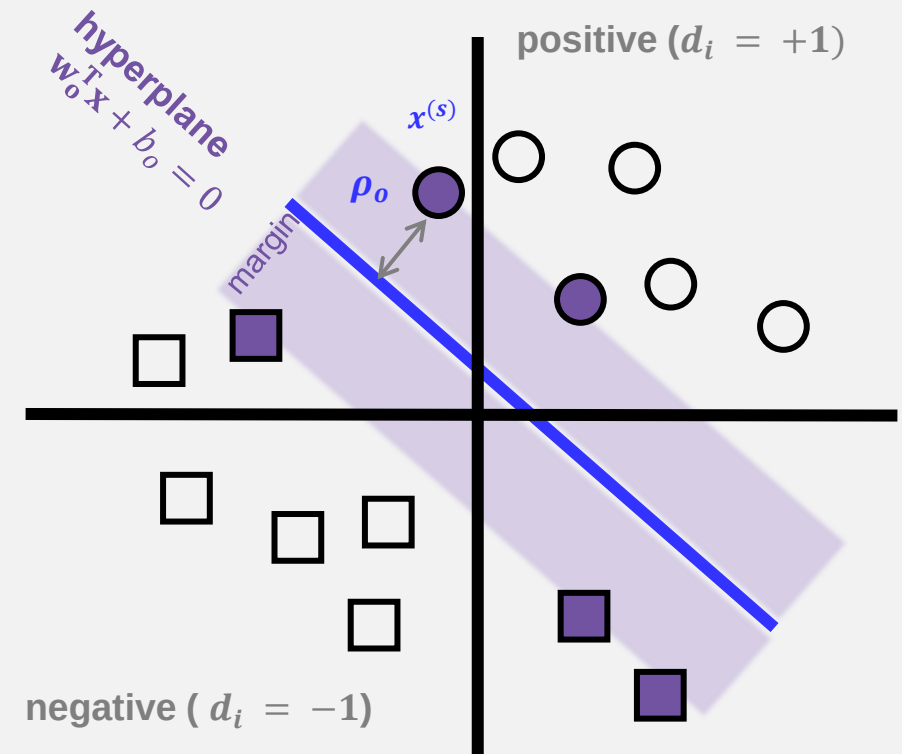
Give a training set $\{(\mathbf{x}_i, d_i)\}_{i=1}^N$, find the optimum values of weight vector $\mathbf{w}$ and b such that they satisfy the constraint

$$d_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 0 \text{ for } i = 1, 2, \dots, N$$

and the weight vector $\mathbf{w}$ minimizes the following cost function (i.e., Euclidean norm of weight)

$$\Phi(\mathbf{w}) = \frac{1}{2} \mathbf{w}^T \mathbf{w}$$

This cost function $\Phi(\mathbf{w})$ is a *convex function*¹ of $\mathbf{w}$ and the constraints are linear in $\mathbf{w}$.



¹See convex function: https://www.princeton.edu/~aaa/Public/teaching/ORF523/S16/ORF523_S16_Lec7_gh.pdf

Support Vector Machine (SVM)

Solving cost function (constrained optimization problem)

$$\Phi(\mathbf{w}) = \frac{1}{2} \mathbf{w}^T \mathbf{w}$$

We construct the Lagrangian function

$$J(\mathbf{w}, b, \alpha) = \frac{1}{2} \mathbf{w}^T \mathbf{w} - \sum_{i=1}^N \alpha_i [d_i(\mathbf{w}^T \mathbf{x}_i + b) - 1]$$

where α is Lagrangian multiplier¹.

¹read details here: <https://www.efzg.unizg.hr/UserDocsImages/John%20Newcastle%20University%20UK%20Lagrange%20Multipliers.pdf>

Support Vector Machine (SVM)

Dual Problem

For such as constrained optimization problem

$J(\mathbf{w}, b, \alpha) = \frac{1}{2} \mathbf{w}^T \mathbf{w} - \sum_{i=1}^N \alpha_i [d_i(\mathbf{w}^T \mathbf{x}_i + b) - 1]$, it is possible to solve another problem called *dual problem* $Q(\alpha)$ that has same optimal values as the first primal problem.

$$Q(\alpha) = \sum_{i=1}^N \alpha_i - \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j d_i d_j \mathbf{x}_i^T \mathbf{x}_j$$

Subject to

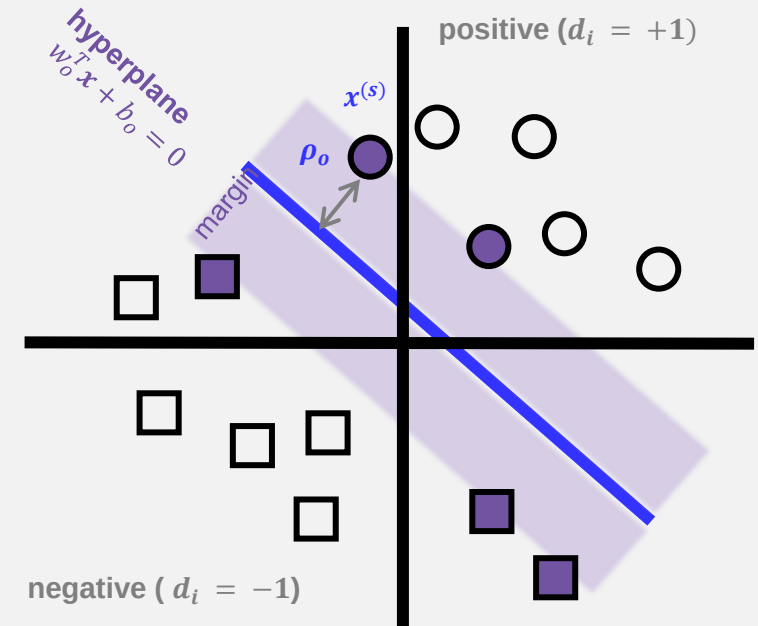
$$1) \sum_{i=1}^N \alpha_i d_i = 0$$

$$2) 0 \leq \alpha_i \leq C \text{ for } i = 1, 2, \dots, N$$

where C is a user defined parameter and optimal weight vector for optimal α_o is

$$\mathbf{w}_o = \sum_{i=1}^{N_s} \alpha_{o,i} d_i \mathbf{x}_i$$

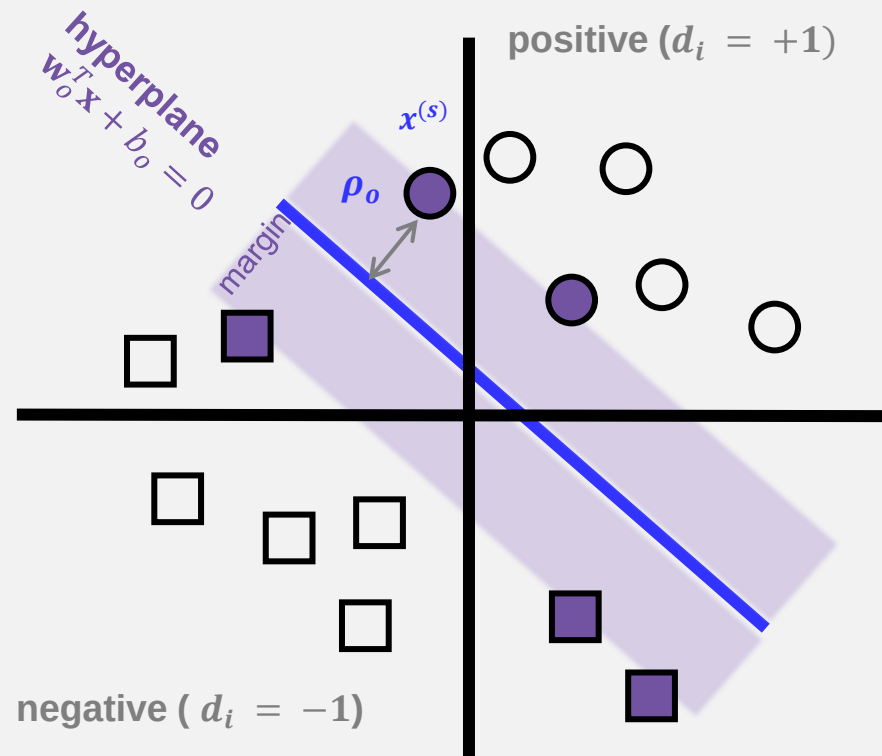
Solve $Q(\alpha)$ using **Sequential minimal optimization (SMO)**



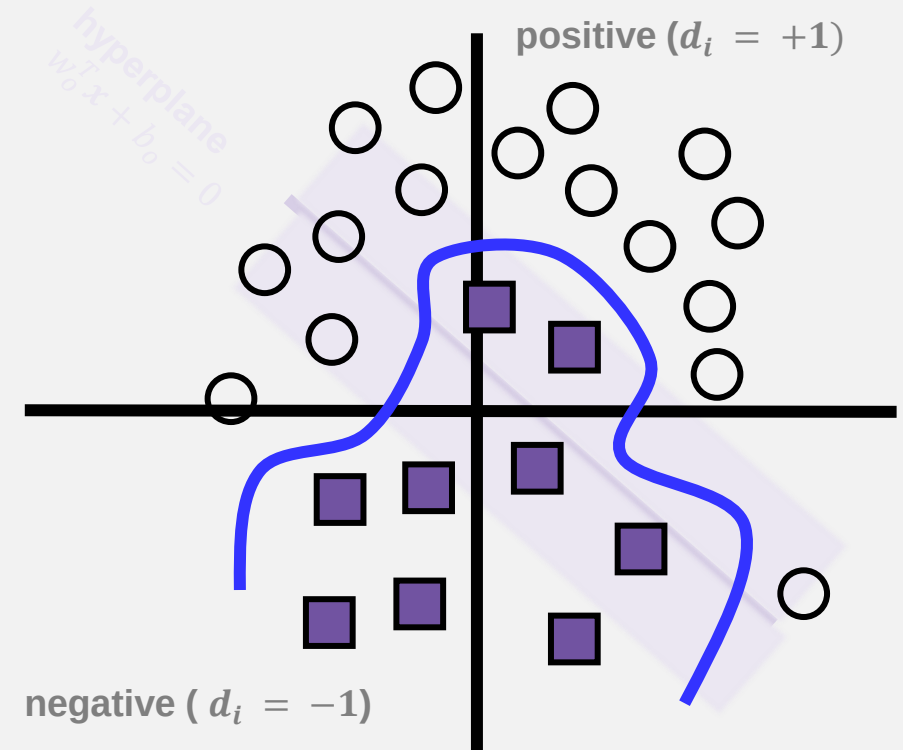
Sequential Minimal Optimization:
A Fast Algorithm for Training
Support Vector Machines John
C. Platt, 1998, Microsoft
Research

Support Vector Machine (SVM)

Linearly Non-separable problems



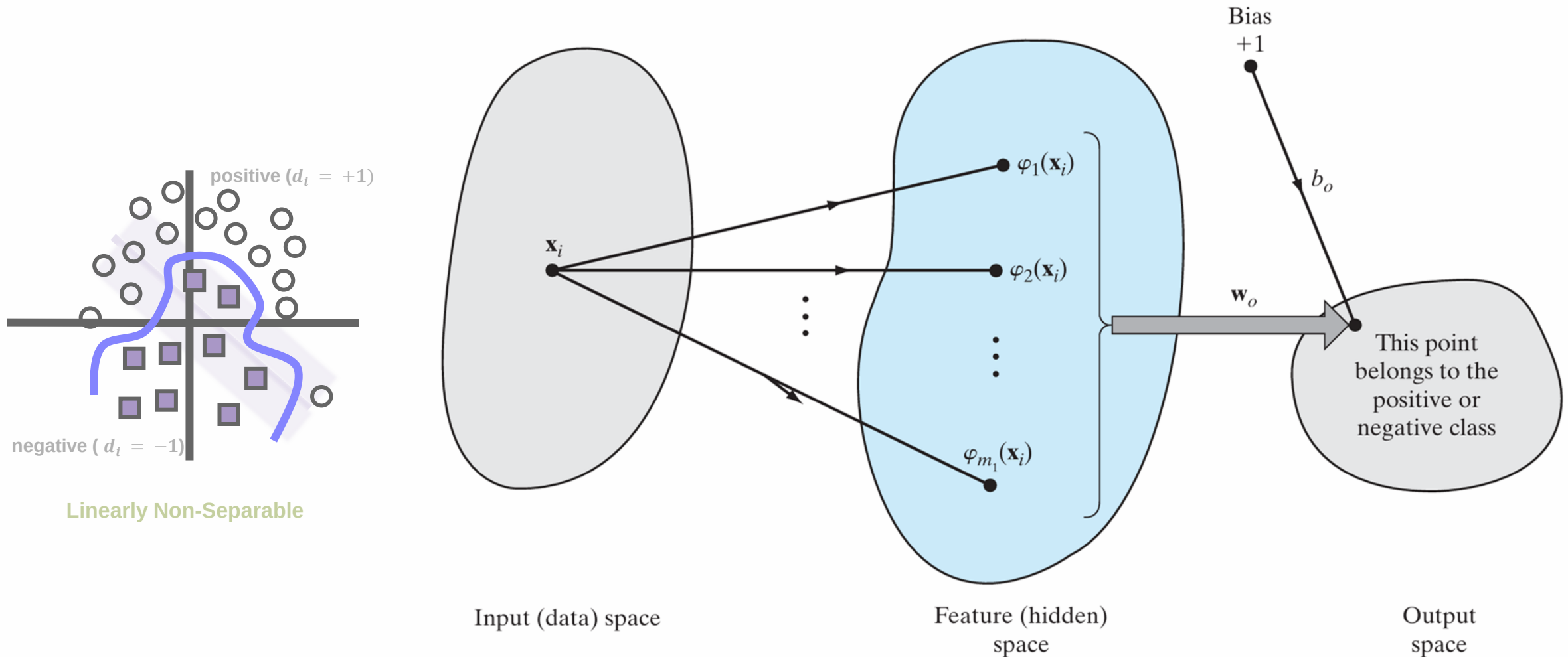
Linearly Separable



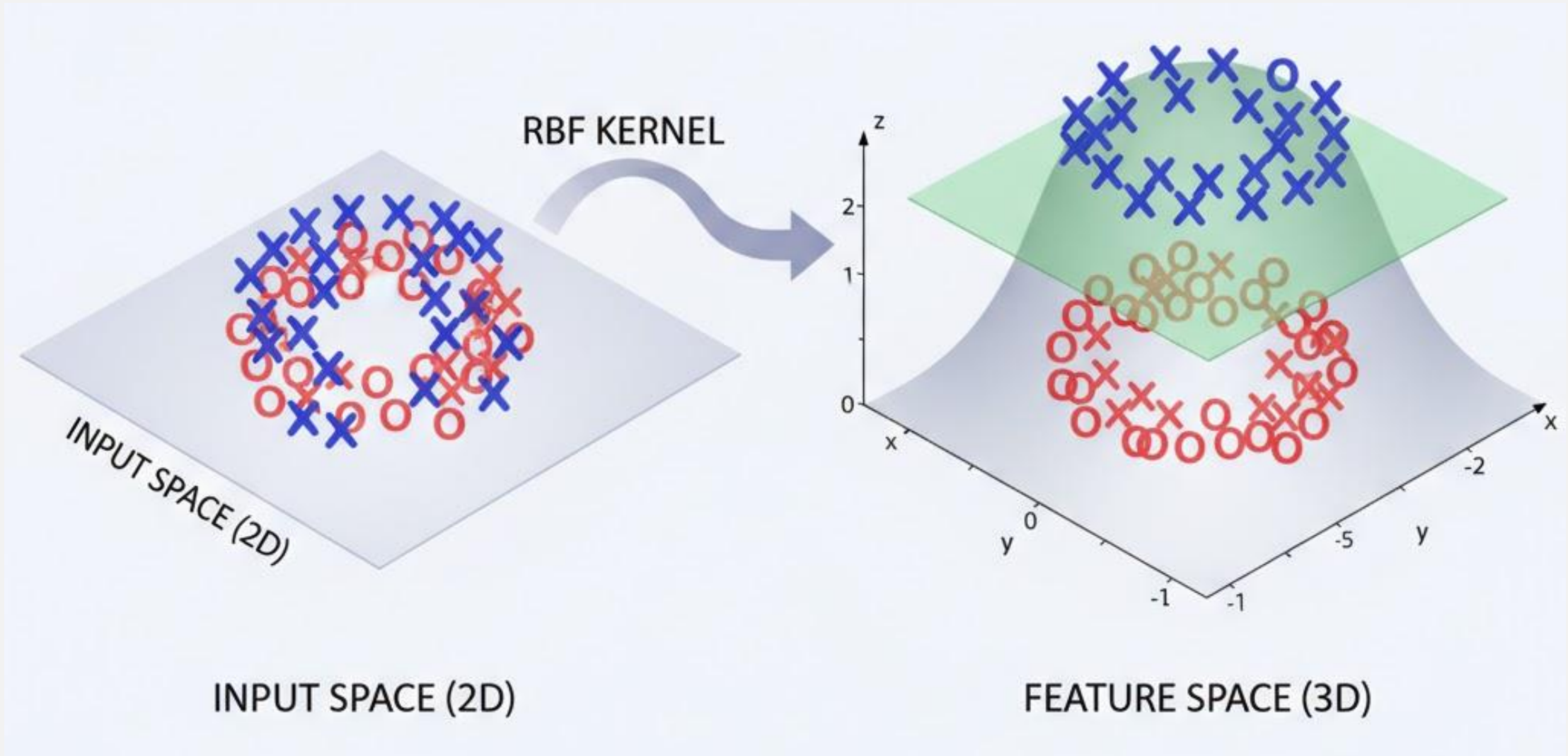
Linearly Non-Separable

Support Vector Machine (SVM)

Linearly Non-separable problems: The Kernel Trick



The Kernel Trick

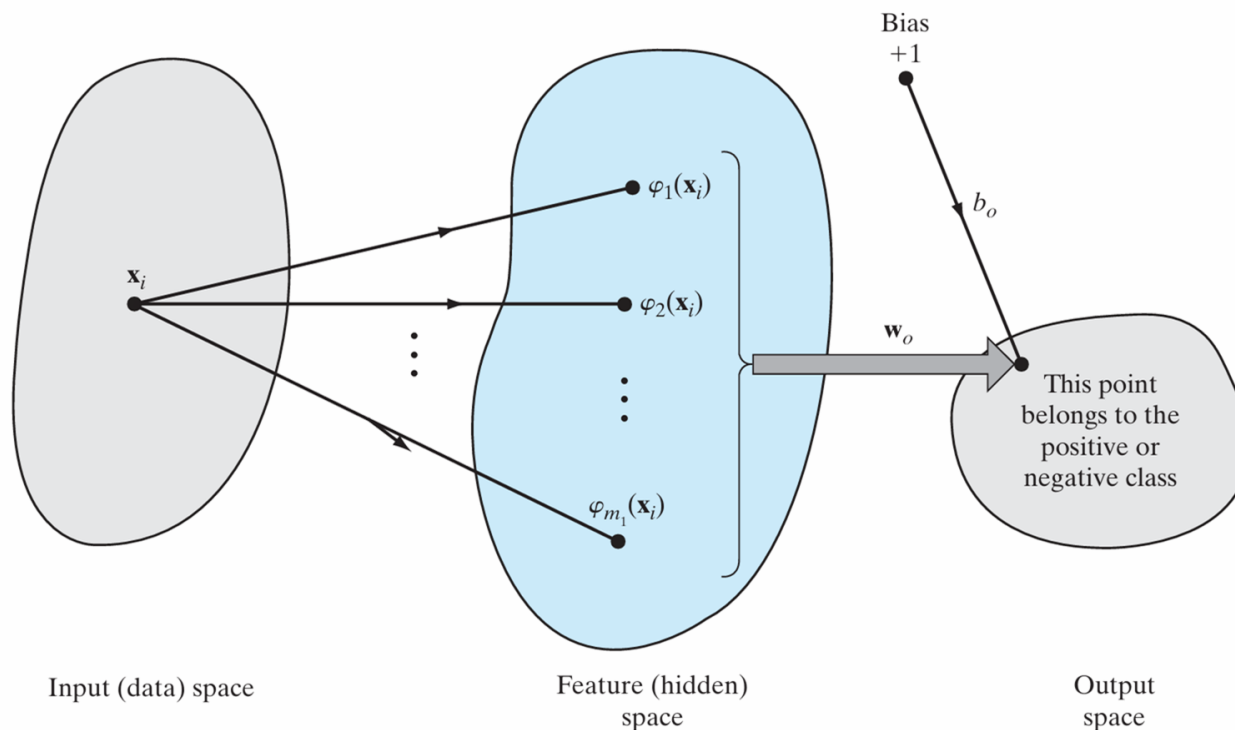


Support Vector Machine as Kernel Machine

The hyperplane that separates these two classes is defined as

$$\mathbf{w}^T \Phi(\mathbf{x}) = 0$$

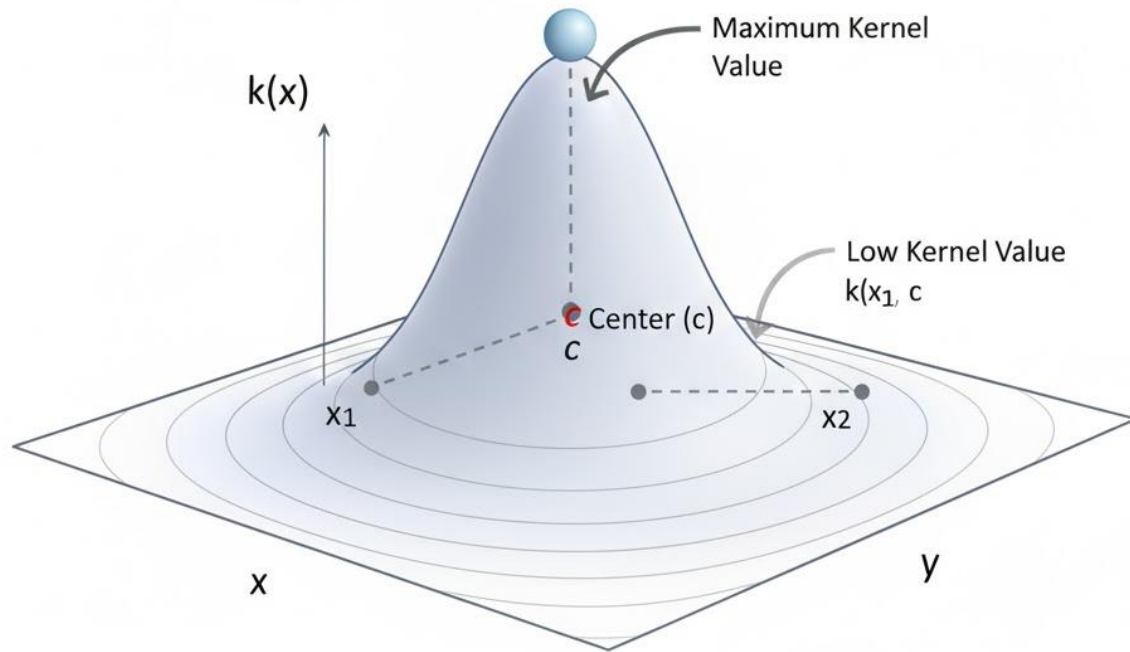
$\mathbf{w}$ is weight associated with projected input to high m dimensional feature space $\Phi(\mathbf{x})$ and $x_0 = b$ is bias.



$$\mathbf{w}_o = \sum_{i=1}^{N_S} \alpha_{o,i} d_i \Phi(\mathbf{x}_i)$$

Kernel Machine

A radial basis function kernel



The hyperplane that separated these two classes is defined as

$$\mathbf{w}^T \Phi(\mathbf{x}) = 0$$

Replacing value $\mathbf{w}$ in above equation we get optimal decision surface

$$\sum_{i=1}^N \alpha_{o,i} d_i \Phi^T(\mathbf{x}_i) \Phi(\mathbf{x})$$

Or

$$\sum_{i=1}^N \alpha_{o,i} d_i k(\mathbf{x}, \mathbf{x}_i)$$

$k(\mathbf{x}, \mathbf{x}_i) = \Phi^T(\mathbf{x}_i) \Phi(\mathbf{x})$ is an inner-product kernel or kernel.

An example kernel could be a radial basis function:

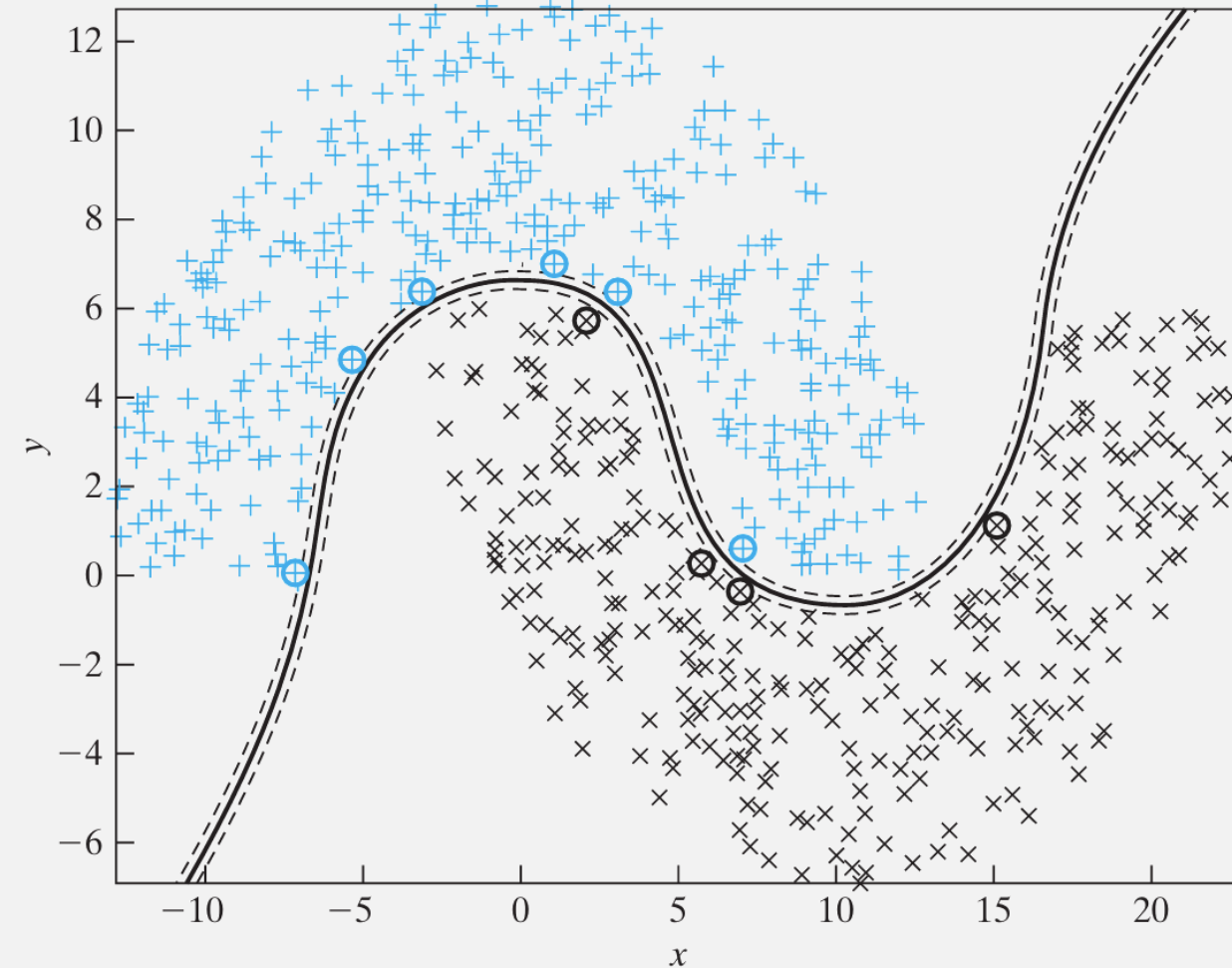
$$k(\mathbf{x}, \mathbf{x}_i) = \exp \left(-\frac{1}{2\sigma^2} \|\mathbf{x} - \mathbf{x}_i\|^2 \right)$$

SVM (Kernel Machine) on some training data

10:29
AM

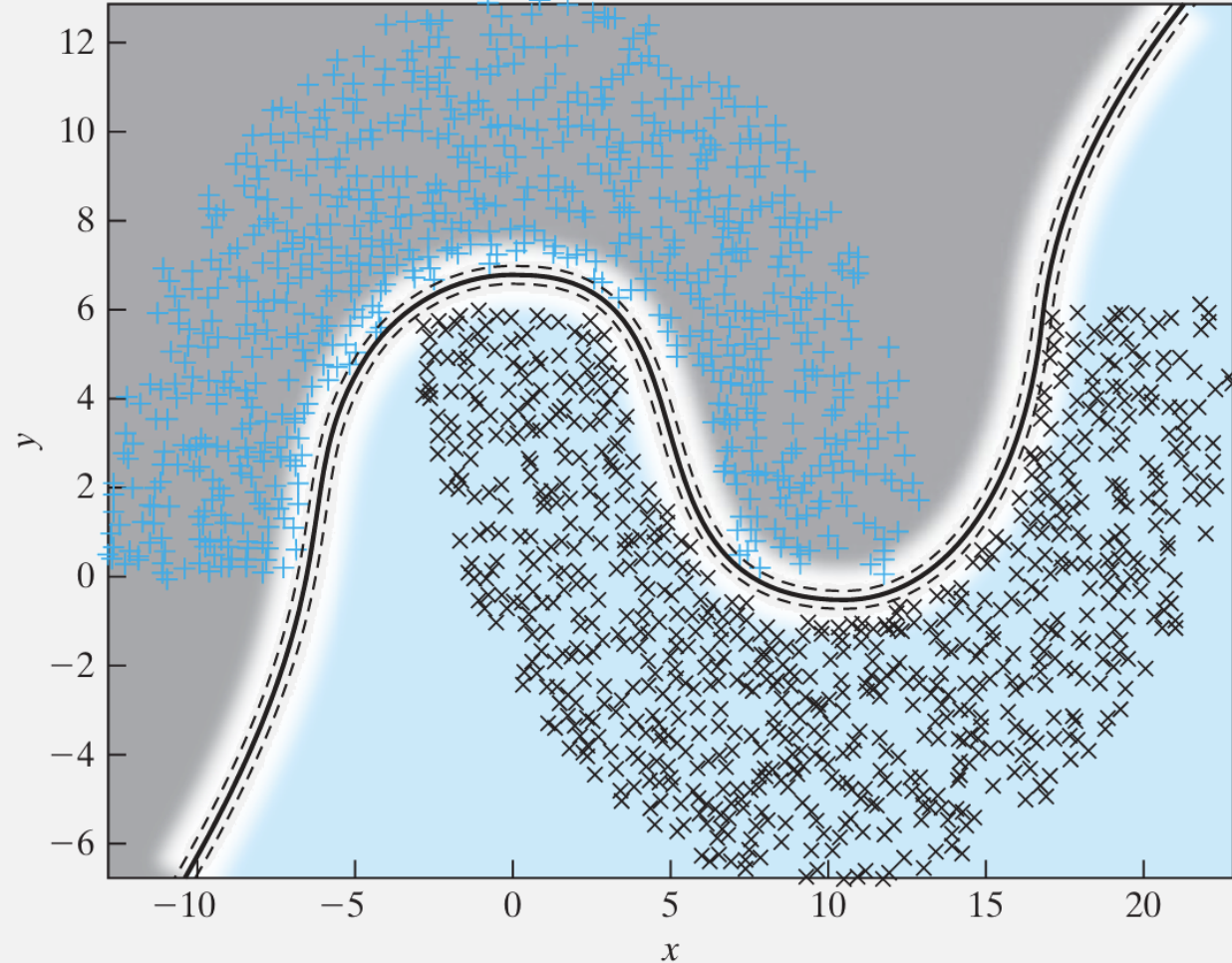
Experiment on SVM for the double-moon data

Classification using SVM with distance = -6, radius = 10, and width = 6

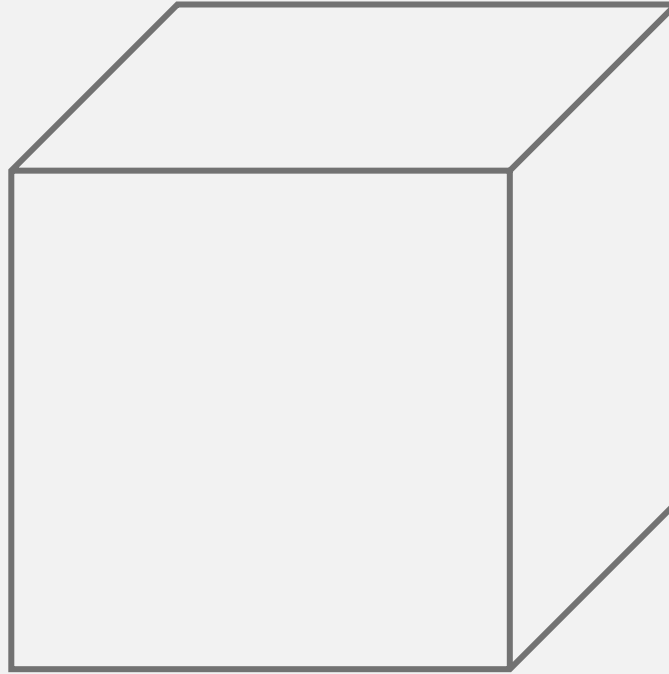


(a) Training result

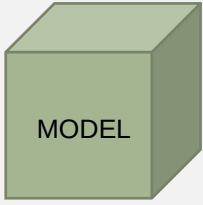
Classification using SVM with distance = -6, radius = 10, and width = 6



(b) Testing result



Model Evaluation



Model Evaluation

Metrics for Performance Evaluation

How to evaluate the performance of a model?

Methods for Performance Evaluation

How to obtain reliable estimates?

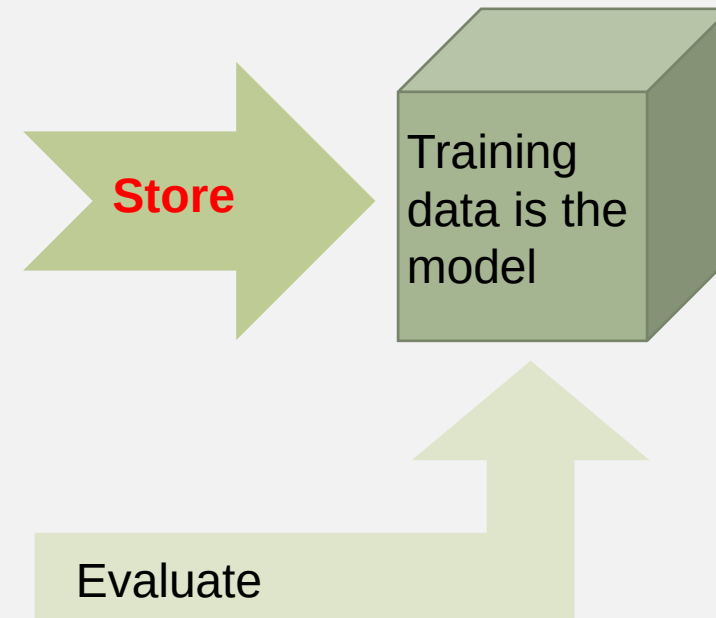
Methods for Model Comparison

How to compare the relative performance among competing models?



Evaluation of Classifiers

	#	X Inputs Attributes (Independent)		Y Target/Class Attributes Dependent
		A1	A2	A3
Training Records (Set)	Ex. 0	A1 ₀	A2 ₀	A3 ₀
	Ex. 1	A1 ₁	A2 ₁	A3 ₁
	Ex. 2	A1 ₂	A2 ₂	A3 ₂
	Ex. 3	A1 ₃	A2 ₃	A3 ₃
	Ex. 4	A1 ₄	A2 ₄	A3 ₄
	Ex. 5	A1 ₅	A2 ₅	A3 ₅
Test Records (Set)	Ex. 6	A1 ₆	A2 ₆	A3 ₆
	Ex. 7	A1 ₇	A2 ₇	A3 ₇
	Ex. 8	A1 ₈	A2 ₈	A3 ₈
	Ex. 9	A1 ₉	A2 ₉	A3 ₉



Evaluating a Classification Model

Classification models are evaluated depending on the correct number of predictions

There are **2 basic categories of classification problems**

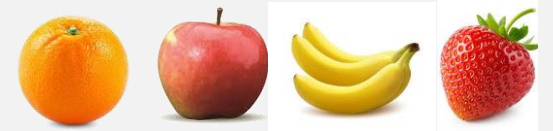
- **Binary classification problem**

- A classification model with only 2 classes



- **Multiclass classification problem**

- A classification model with more than 2 classes



Misclassification rate, E

Average error of wrong classification, best performance is value 0% and worst performance is value is 100%

$$E = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i \neq y_i)$$


$\hat{y}_i$ - predicted output

y_i - target output

n - number of examples in training/test set

Accuracy, A

Average correct classification, best performance is value 100% and worst performance is value is 0%

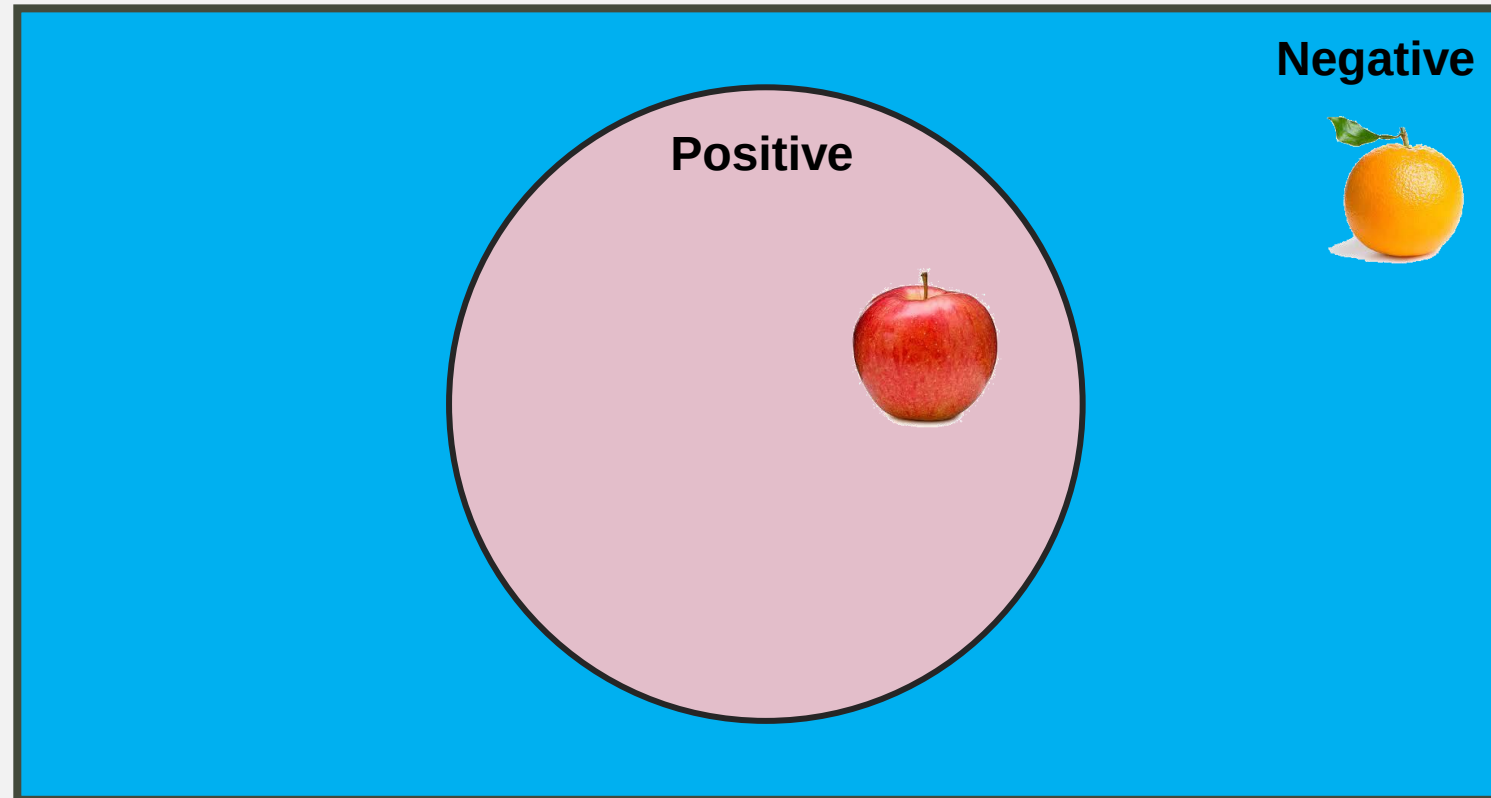
$$A = \frac{1}{n} \sum_{i=1}^n \overset{\text{🍏}}{(\hat{y}_i = y_i)} \overset{\text{🍏}}$$

$\hat{y}_i$ - predicted output

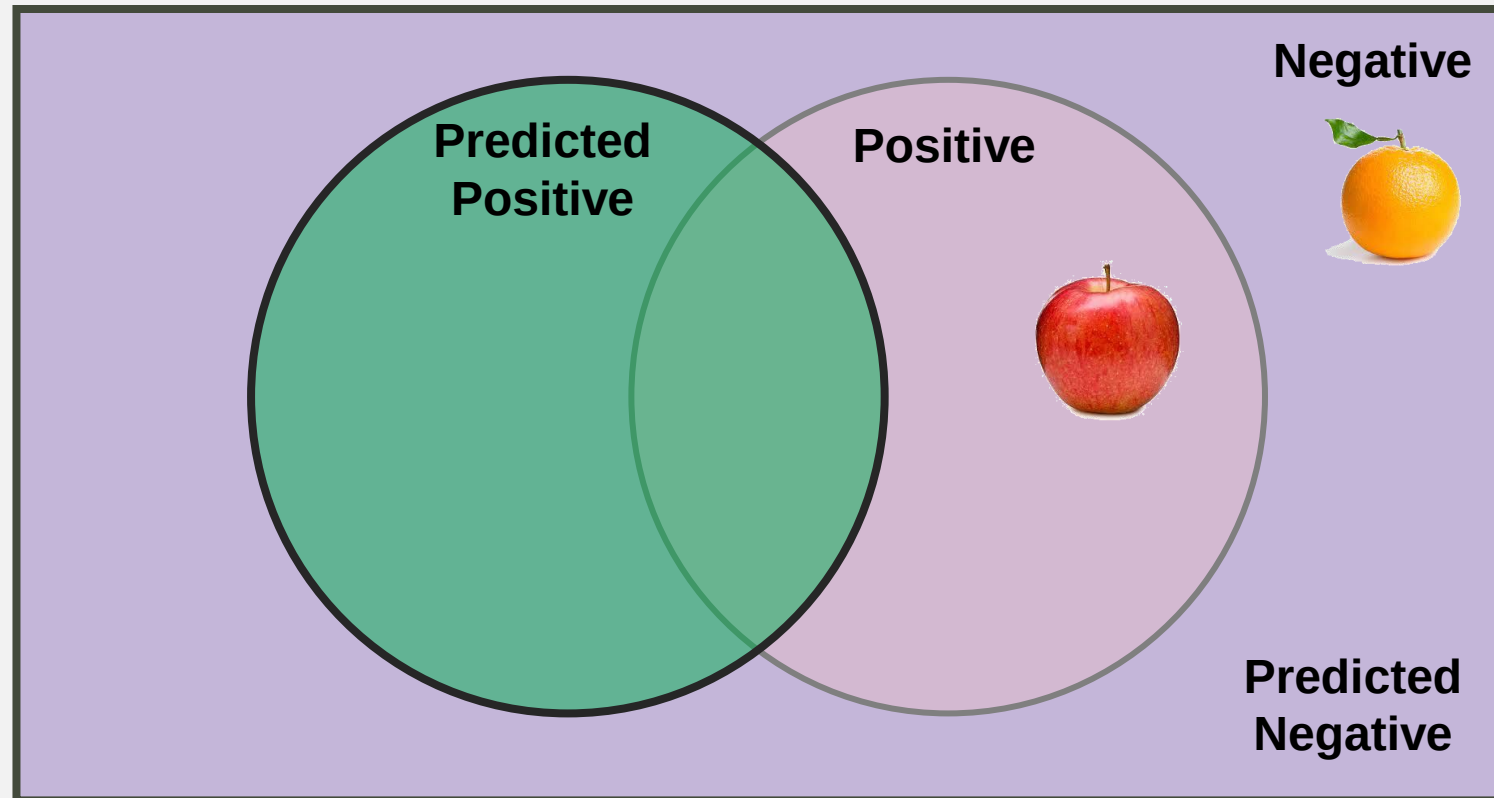
y_i - target output

n - number of examples in training/test set

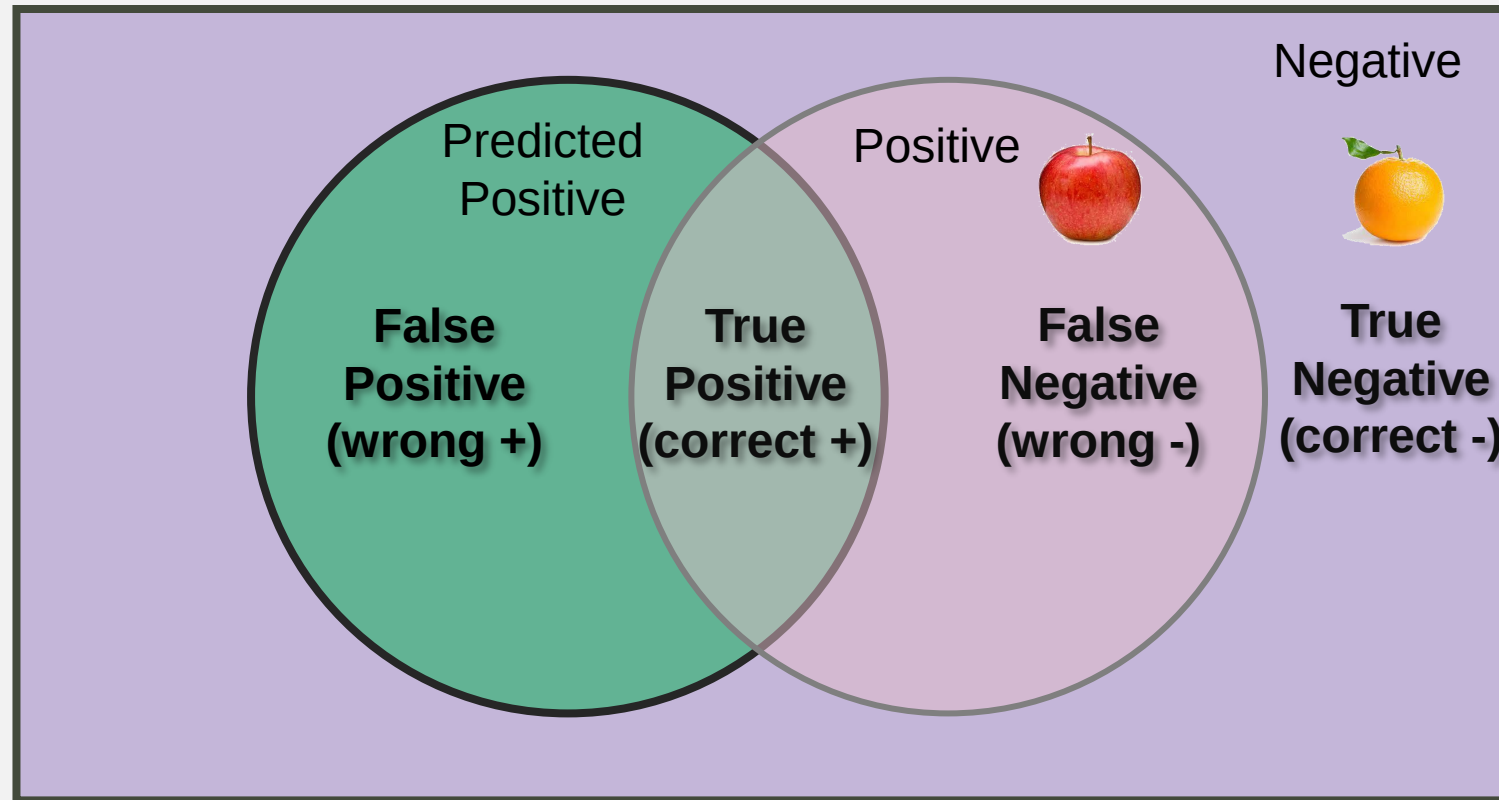
A Binary Classification Problem



Prediction of a Classifier



True Positive (correct +), True Negatives (correct -),
False Positive (wrong +), False Negatives (wrong -),



A better way to display results of predictions is as a **confusion matrix** (A matrix of actual data vs predicted data). Many more metrics can be generated according to values in the confusion matrix.

The Confusion Matrix

(A matrix of actual data vs predicted data)

	Predicted Class 0 	Predicted Class 1 
Actual Class 0 	True Positive (TP) <i>(apple correctly predicted as apple)</i>	False Negative (FN) <i>(apple wrongly predicted as orange)</i>
Actual Class 1 	False Positive (FP) <i>(orange wrongly predicted as apple)</i>	True Negative (TN) <i>(orange correctly predicted as orange)</i>

From the confusion matrix the total accuracy of the model can be generated:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

The accuracy states how many predictions were correct



Accuracy: Limitation

Consider a 2-class problem

Number of Class 0 examples = 9990



Number of Class 1 examples = 10

If model predicts everything to be **class 0**, 

Model's Accuracy is $9990/10000 = 99.9\%$

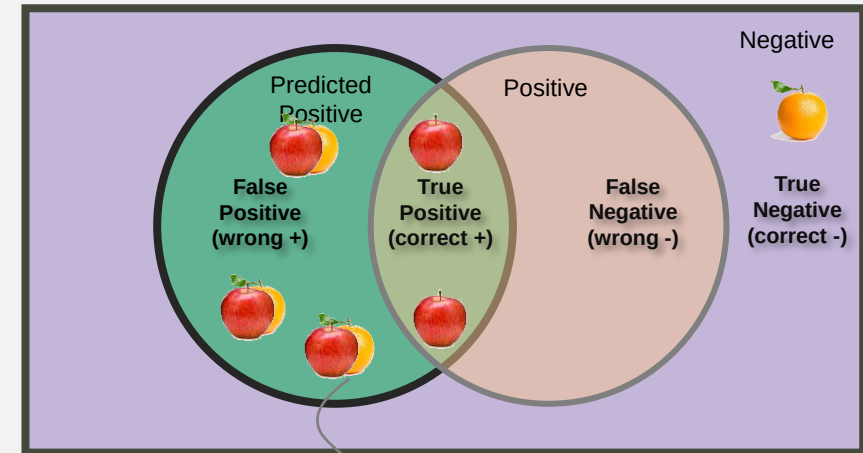
Hence, accuracy is misleading because, the model does not detect any **class 1**  example

Evaluation Metrics: Precision

Precision is the fraction of positive predictions for the respective class that are correct: *How well you guess (apple as apple)*

$$\text{Precision} = \frac{\text{True Positive}}{\text{True Positive} + \text{False Positive}}$$

$$= \frac{\text{Total apples correctly predicted as apples}}{\text{Total number of positive prediction in the data}}$$



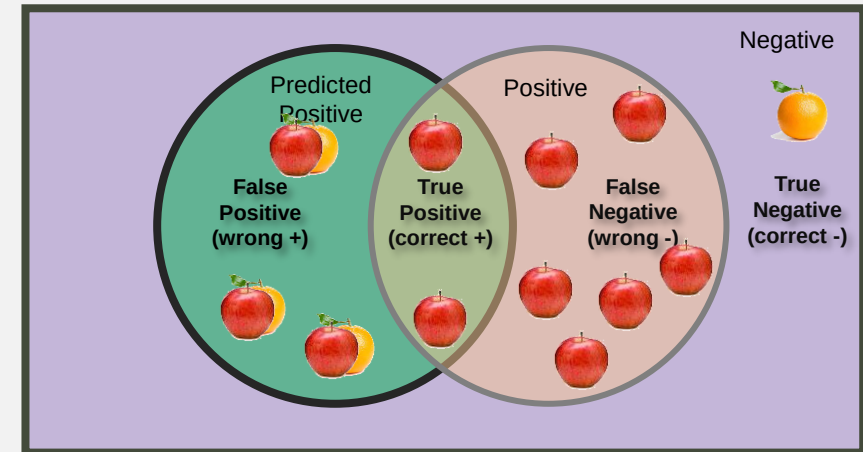
It's overlapping because actual data underneath is orange but predicted as an apple

Evaluation Metrics: Recall

Recall is the fraction of positive values in the data that we correctly predict: *How complete is the prediction?*

$$\text{Recall} = \frac{\text{True Positive}}{\text{True Positive} + \text{False Negative}}$$

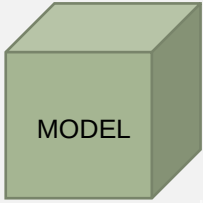
$$= \frac{\text{Total apples correctly predicted as apples}}{\text{Total number of positives (apples) in the data}}$$



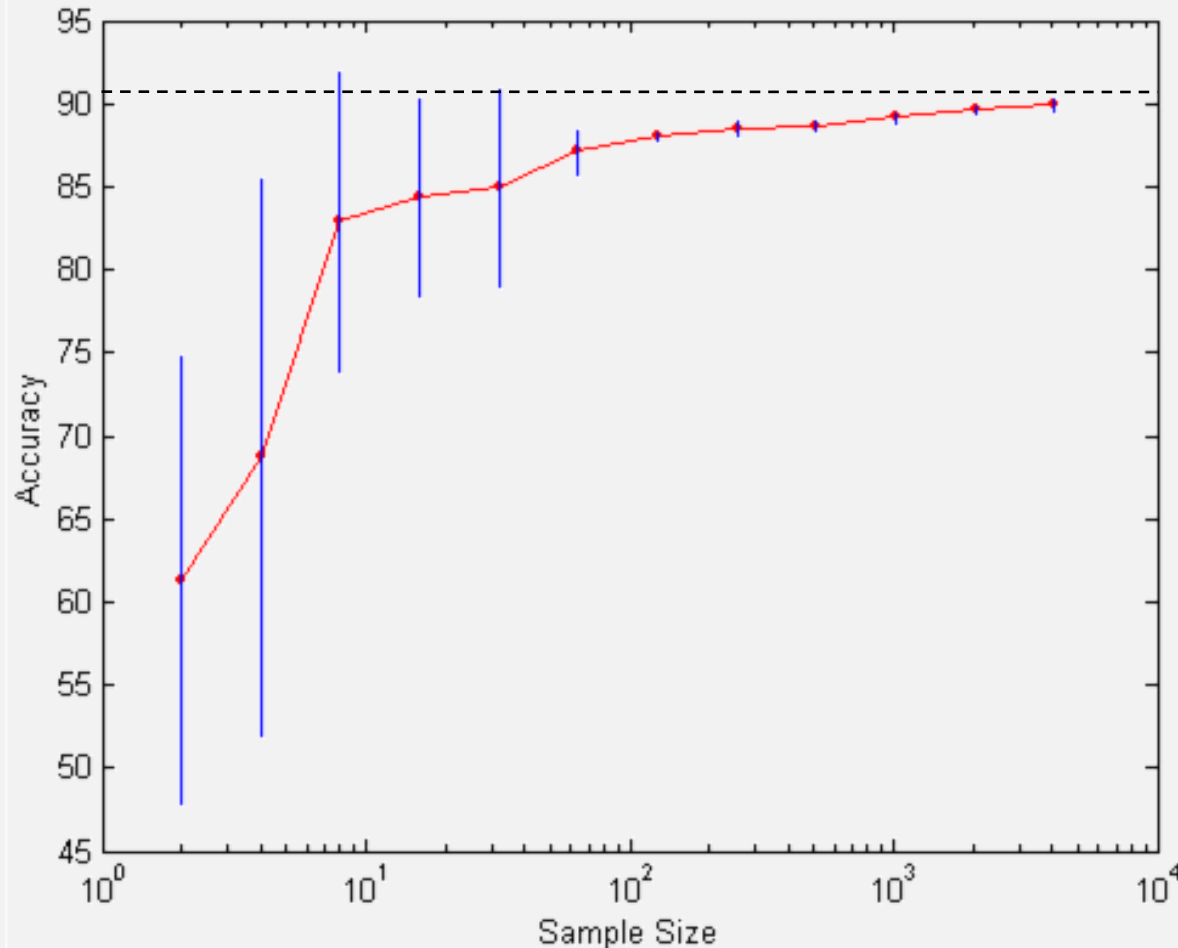
Trade off between Precision and Recall

- When dealing with real world data you will never see 100% precision and recall at the same time
- For example: **if all the samples are predicted as a positive class in a dataset you will get 100 % recall but a low precision statistic**
- Instead, the F1 score is used to generate a single metric that balances the precision and recall:

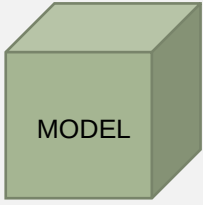
$$\text{F1 Score} = 2 * \frac{\text{Precision} * \text{Recall}}{\text{Precision} + \text{Recall}}$$



Method for Performance Evaluation

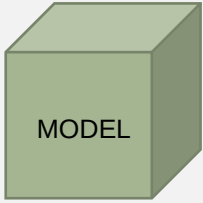


- Learning curve shows how accuracy changes with varying sample size
- Requires a sampling schedule for creating learning curve:
 - Arithmetic sampling (Langley, et al)
 - Geometric sampling (Provost et al)
- Effect of small sample size:
 - Bias in the estimate
 - Variance of estimate



Method of Estimation

- **Holdout**
 - Original data is partitioned into two disjoint sets
- **Random subsampling**
 - Repeated holdout
 - Still some issues
 - Similar issues to holdout
 - No control over the selected samples (samples may be chosen multiple times or never)
- **Cross validation**
 - Partition data into k disjoint subsets
 - k -fold: train on $k-1$ partitions, test on the remaining one
 - Leave-one-out: $k=n$
- **Bootstrap**
 - Sampling with replacement

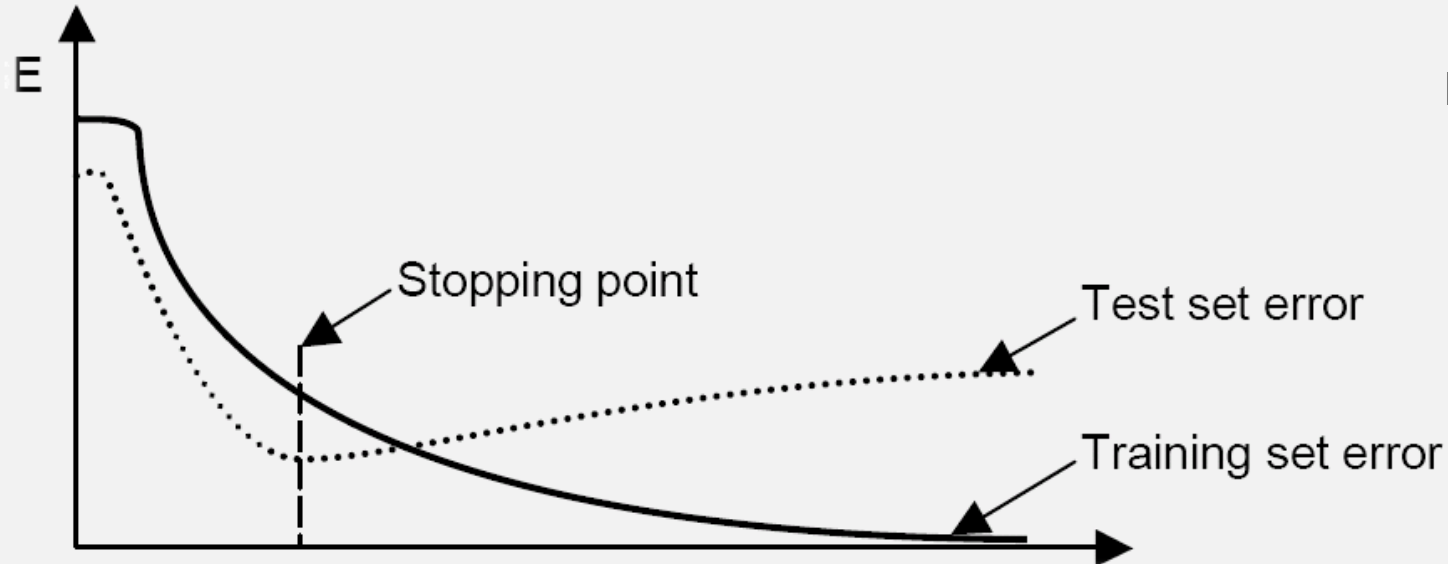
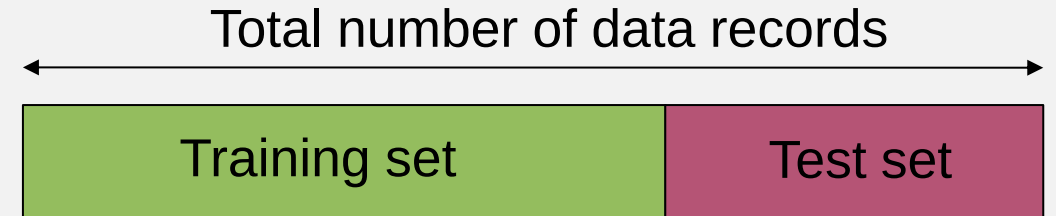


Holdout Method

Split dataset into two groups

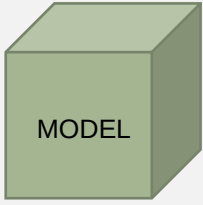
Training set: used to train the classifier

Test set: used to estimate the error rate of the trained classifier



Limitations

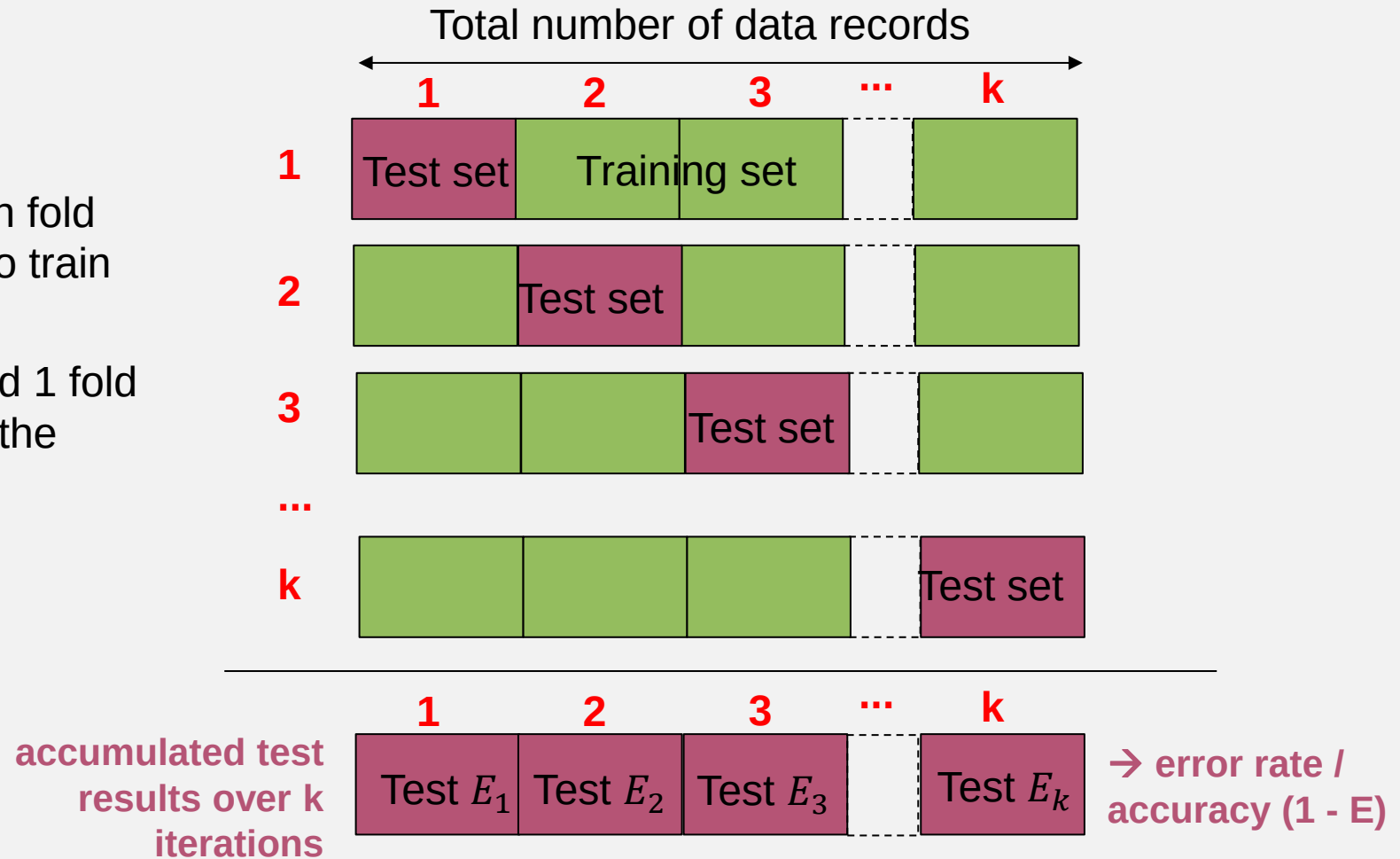
- Less data available for training
- Bias on the training set
- Training set and test set are not independent (e.g. there may be unbalanced classes in both)

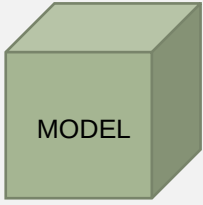


Cross Validation

- k-fold cross validation**

- Training set: at each fold $k-1$ folds are used to train a classifier
- Test set: at each fold 1 fold is used to estimate the error rate

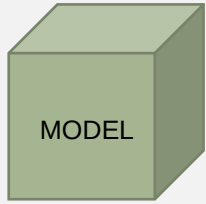




Model Comparison

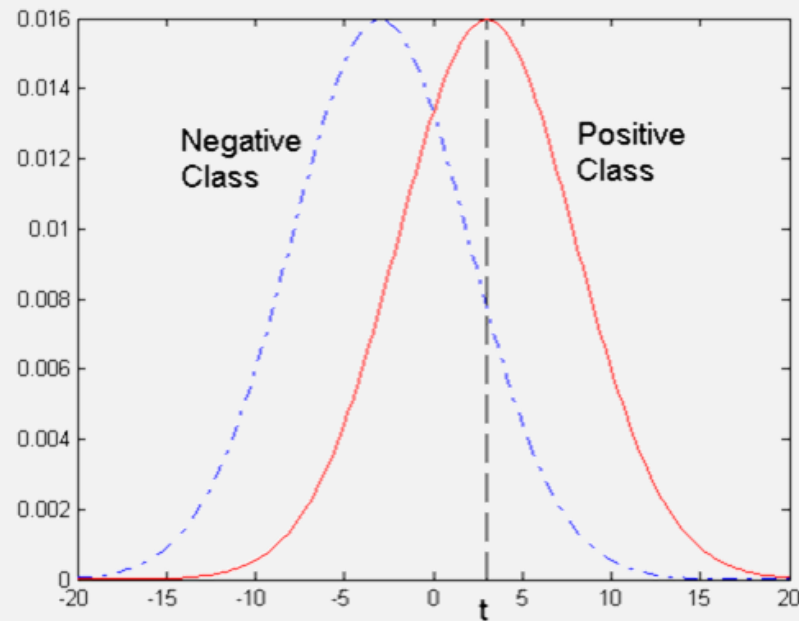
ROC (Receiver Operating Characteristic)

- Developed in 1950s for signal detection theory to analyze noisy signals
 - Characterize the trade-off between positive hits and false alarms
- ROC curve plots True Positive (on the y-axis) against False Positive (on the x-axis)
- Performance of each classifier represented as a point on the ROC curve
 - changing the threshold of algorithm, sample distribution changes the location of the point



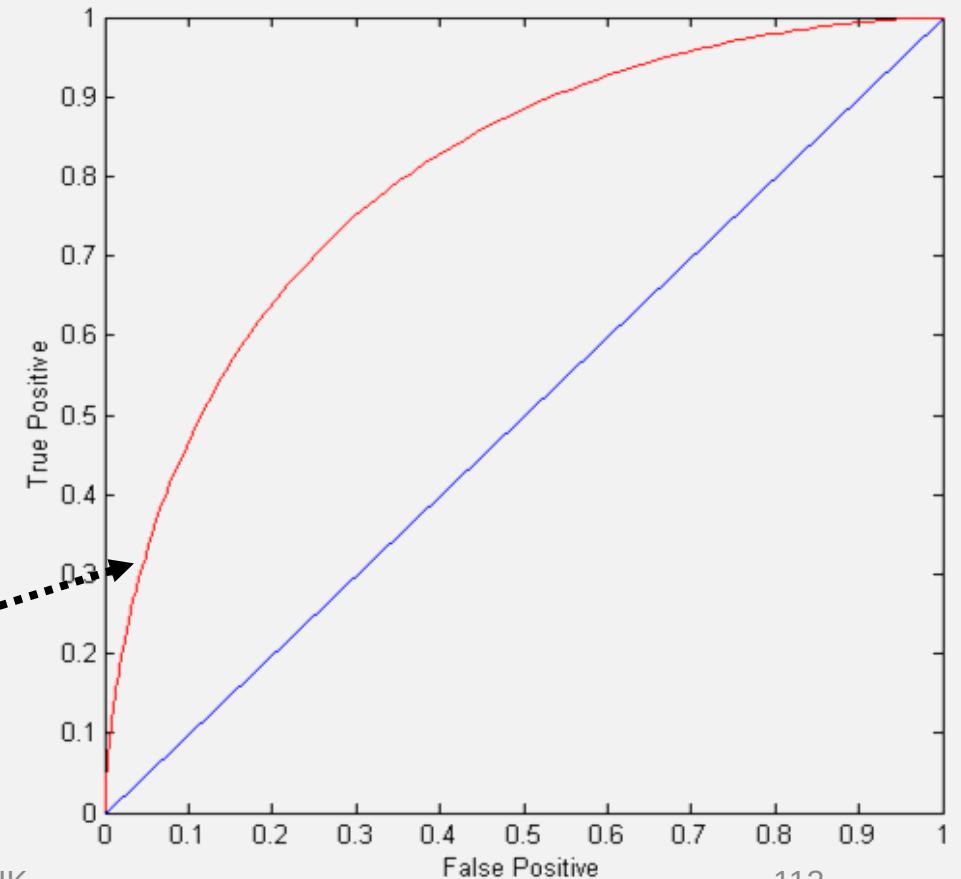
ROC (Receiver Operating Characteristic)

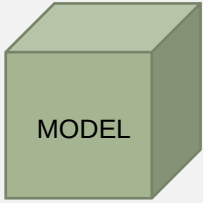
- 1-dimensional data set containing 2 classes (positive and negative)
- any points located at $x > t$ is classified as positive



At threshold t :

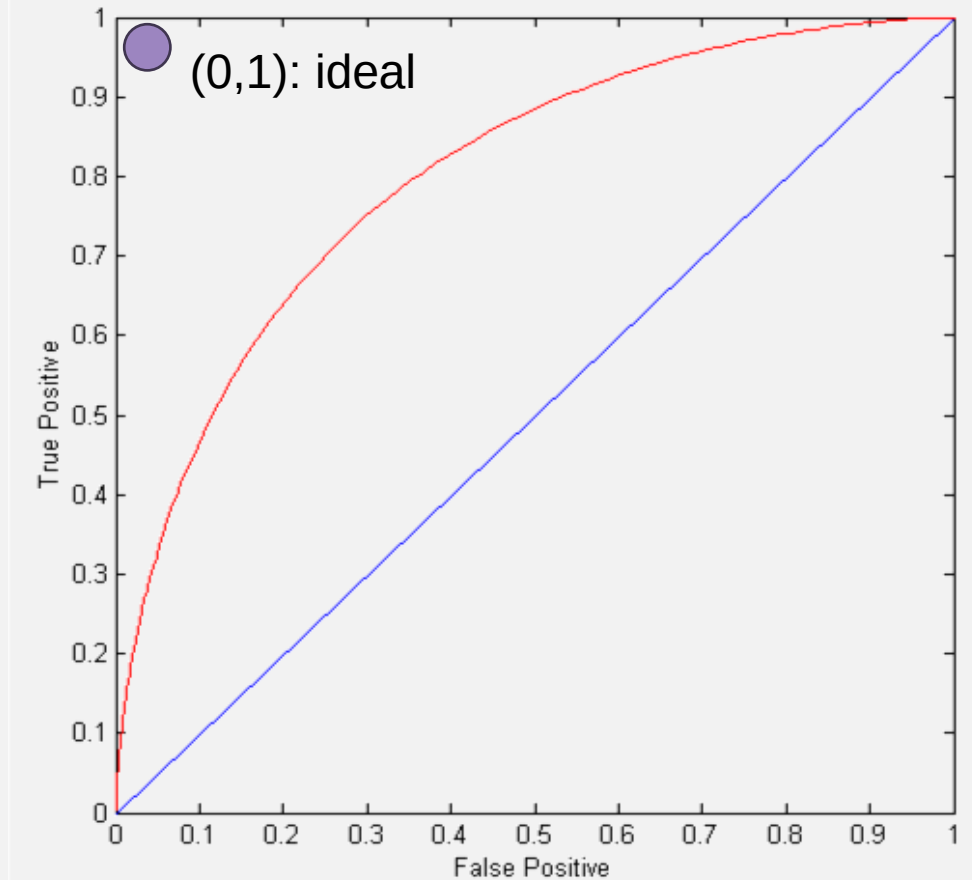
TP=0.5, FN=0.5, FP=0.12, FN=0.88

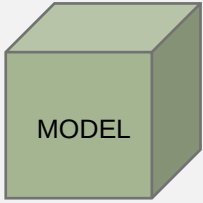




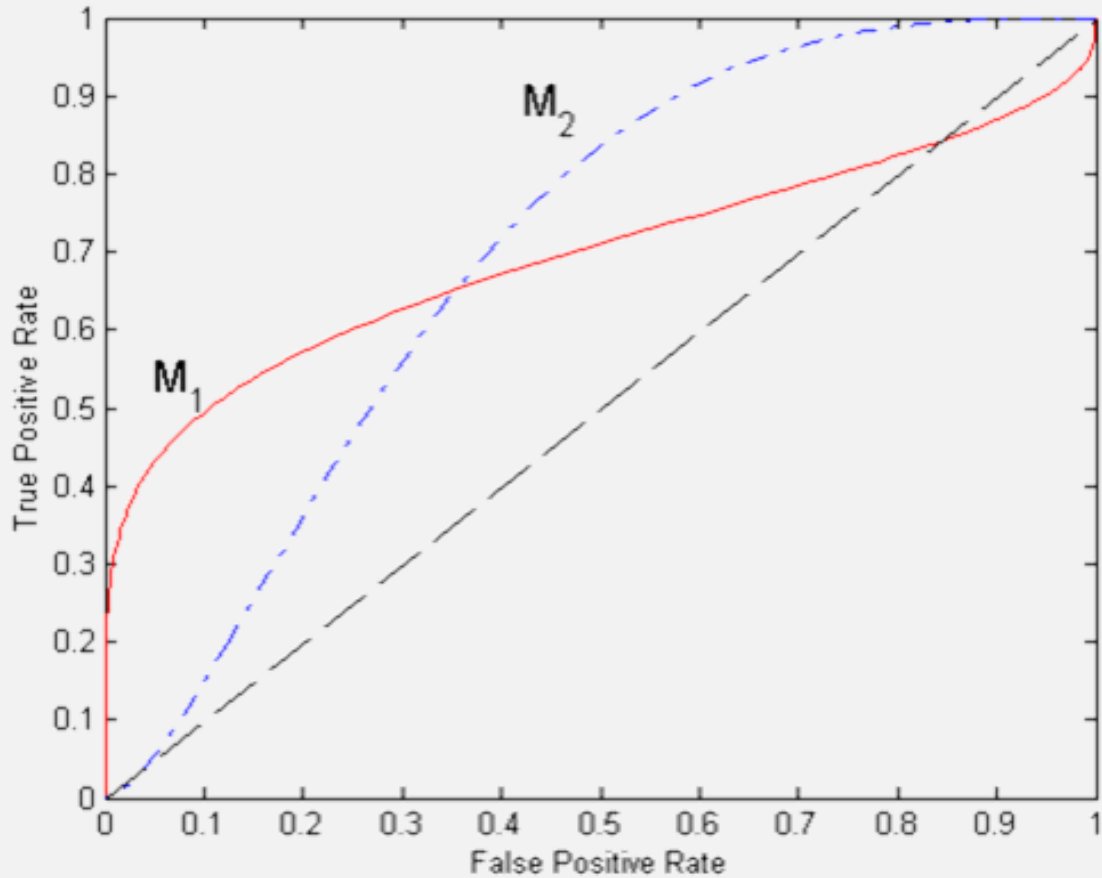
ROC (Receiver Operating Characteristic)

- (TP,FP):
- (0,0): declare everything to be negative class
- (1,1): declare everything to be positive class
- (0,1): ideal
- Diagonal line:
 - Random guessing
 - Below diagonal line:
 - prediction is opposite of the true class

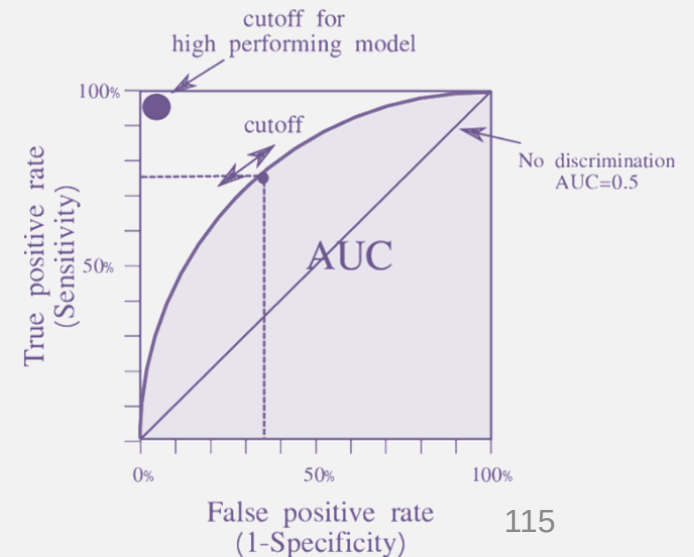




ROC (Receiver Operating Characteristic)



- No model consistently outperform the other
 - M_1 is better for small FPR
 - M_2 is better for large FPR
- Area Under the ROC curve
 - Ideal:
 - Area = 1
 - Random guess:
 - Area = 0.5



Part 4

Practical Session

See attached document in Canvas

